AI-POWERED CULINARY ASSISTANT: REAL-TIME INGREDIENT RECOGNITION WITH BUDGET AND CALORIE-OPTIMIZED MEAL PLANNING

MUHAMMAD NUR IRFAN IZDIYAT BIN MOHD NAZRI

UNIVERSITI TEKNIKAL MALAYSIA MELAKA

BORANG PENGESAHAN STATUS LAPORAN

JUDUL**:** AI-POWERED CULINARY ASSISTANT: REAL-TIME INGREDIENT RECOGNITION WITH BUDGET AND CALORIE-OPTIMIZED MEAL PLANNING

SESI PENGAJIAN: 2025 / 2026

Saya: MUHAMMAD NUR IRFAN IZDIYAT BIN MOHD NAZRI

mengaku membenarkan tesis Projek Sarjana Muda ini disimpan di Perpustakaan Universiti Teknikal Malaysia Melaka dengan syarat-syarat kegunaan seperti berikut:

1. Tesis dan projek adalah hakmilik Universiti Teknikal Malaysia Melaka.
2. Perpustakaan Fakulti Kecerdasan Buatan dan Keselamatan Siber dibenarkan membuat salinan unituk tujuan pengajian sahaja.
3. Perpustakaan Fakulti Kecerdasan Buatan dan Keselamatan Siber dibenarkan membuat salinan tesis ini sebagai bahan pertukaran antara institusi pengajian tinggi.
4. * Sila tandakan (✓)

_____	SULIT	(Mengandungi maklumat yang berdarjah keselamatan atau kepentingan Malaysia seperti yang termaktub di dalam AKTA RAHSIA RASMI 1972)
_____	TERHAD	(Mengandungi maklumat TERHAD yang telah ditentukan oleh organisasi / badan di mana penyelidikan dijalankan)
____✓___	TIDAK TERHAD	

(TANDATANGAN PELAJAR)

(TANDATANGAN PENYELIA)

Alamat tetap: A-03-10, Jalan Anggerik Villa 2/2, Taman Anggerik Villa 2, 43000 Kajang, Selangor

Ts. AHMAD FADZLI NIZAM BIN ABDUL RAHMAN

AI-POWERED CULINARY ASSISTANT: REAL-TIME INGREDIENT RECOGNITION WITH BUDGET AND CALORIE-OPTIMIZED MEAL PLANNING

MUHAMMAD NUR IRFAN IZDIYAT BIN MOHD NAZRI

This report is submitted in partial fulfillment of the requirements for the Bachelor of Computer Science (Artificial Intelligence) with Honours.

FACULTY OF ARTIFICIAL INTELLIGENCE AND CYBER SECURITY
UNIVERSITI TEKNIKAL MALAYSIA MELAKA

2026

DECLARATION

I hereby declare that this project report entitled

AI-POWERED CULINARY ASSISTANT: REAL-TIME INGREDIENT RECOGNITION WITH BUDGET AND CALORIE-OPTIMIZED MEAL PLANNING

is written by me and is my own effort and that no part has been plagiarized without citations.

STUDENT : _____ Date : 17/06/2026
(MUHAMMAD NUR IRFAN IZDIYAT BIN MOHD NAZRI)

I hereby declare that I have read this project report and found this project report is sufficient in term of the scope and quality for the award of Bachelor of Computer Science (Artificial Intelligence) with Honours.

SUPERVISOR : _____ Date : 19/06/2026
(Ts. AHMAD FADZLI NIZAM BIN ABDUL RAHMAN)

DEDICATION

This research project is specially dedicated to my beloved parents, for their endless love, continuous prayers, and countless sacrifices that have brought me to where I am today. Their unwavering support and encouragement have been my greatest source of strength and motivation throughout my entire educational journey.

I would also like to dedicate this work to my esteemed supervisor, Ts. Ahmad Fadzli Nizam bin Abdul Rahman, for his continuous guidance, patience, and invaluable insights throughout the development of this intelligent system. Furthermore, this dedication extends to all my supportive friends and fellow course mates who have generously shared their knowledge and stood by me through the challenges and triumphs of this academic endeavor.

ACKNOWLEDGEMENTS

To begin, I would like to express my heartfelt gratitude to Ts. Ahmad Fadzli Nizam bin Abdul Rahman, my project supervisor, for his extensive advice and assistance during this project. His feedback and suggestions, as well as his professional experience, have helped guide the completion of The Fridge Chef: AI-Powered Culinary Assistant.

I would like to thank all of the instructors and personnel at the Faculty of Artificial Intelligence and Cyber Security (FAIX). They fostered strong intellectual and technical abilities, which considerably benefited the system's development. Their genuine passion for knowledge inspires others.

Parents are often supportive of their children's academic performance, but mine went above and beyond by providing unconditional love, which acted as motivation. Having regular encouragement, combined with prayer, helped me remain psychologically strong during the most difficult periods of my journey.

Many students put off their projects till the last minute; nevertheless, my classmates provided emotional support while also assisting me in brainstorming for various components through meaningful discussions about this work. In addition, having friends nearby made everything feel a lot better.

Your support as well as motivation throughout every stage has been impactful where words cannot describe how grateful I truly am therefore I thank each individual that helped implement parts into this project directly or indirectly.

ABSTRACT

This project presents "The Fridge Chef," an AI-powered culinary assistant designed to address the interconnected challenges of strict financial constraints, household food waste, and the lack of localized, nutritionally aware meal planning tools for university students and individuals managing strict financial constraints. The primary objective is to develop a seamless Progressive Web Application (PWA) that integrates Computer Vision (CV) and Natural Language Processing (NLP) to dynamically generate budget-optimized and culturally appropriate recipes. The methodology employs a custom-trained Convolutional Neural Network (CNN) to accurately recognize a localized dataset of 22 common Malaysian food ingredients from user-provided images, eliminating the friction of manual inventory tracking. Subsequently, the system utilizes a Large Language Model (LLaMA-3 via the Groq API) to synthesize detailed cooking protocols constrained by user-defined caloric limits and Halal dietary requirements. To ensure precise financial accuracy and prevent algorithmic arithmetic hallucination, the architecture integrates real-time median retail prices from the Malaysia Open Data Portal (OpenDOSM), utilizing deterministic frontend logic for exact cost aggregation. Developed through an Agile software development life cycle, the resulting prototype successfully processes visual ingredient inputs and outputs comprehensive, health-conscious, and traditional Malaysian state recipes. The implementation demonstrates that combining visual recognition with LLM-driven recommendation engines can effectively empower users to optimize daily food expenditures, improve nutritional tracking, and significantly reduce domestic food waste through intelligent leftover utilization.

ABSTRAK

Projek ini membentangkan "The Fridge Chef," sebuah pembantu masakan berkuasa AI yang direka untuk menangani cabaran yang saling berkaitan seperti kekangan kewangan yang ketat, pembaziran makanan isi rumah, dan kekurangan alat perancangan makanan tempatan yang peka nutrisi untuk pelajar universiti dan individu yang menguruskan had kewangan yang ketat. Objektif utama adalah untuk membangunkan Aplikasi Web Progresif (PWA) yang lancar, yang mengintegrasikan Penglihatan Komputer (CV) dan Pemprosesan Bahasa Asli (NLP) untuk menjana resipi yang dioptimumkan bajet dan bersesuaian dengan budaya secara dinamik. Metodologi ini menggunakan Rangkaian Neural Konvolusi (CNN) yang dilatih khas untuk mengecam set data tempatan yang terdiri daripada 22 bahan makanan biasa Malaysia secara tepat daripada imej yang diberikan oleh pengguna, sekaligus menghapuskan kesukaran penjejakan inventori secara manual. Seterusnya, sistem ini menggunakan Model Bahasa Besar (LLaMA-3 melalui API Groq) untuk mensintesis protokol memasak terperinci yang dikekang oleh had kalori yang ditetapkan pengguna dan keperluan diet Halal. Untuk memastikan ketepatan kewangan yang jitu dan mencegah halusinasi aritmetik algoritma, seni bina sistem ini mengintegrasikan harga runcit median masa nyata daripada Portal Data Terbuka Malaysia (OpenDOSM), menggunakan logik bahagian hadapan deterministik untuk pengagregatan kos yang tepat. Dibangunkan melalui kitaran hayat pembangunan perisian Agile, prototaip yang dihasilkan berjaya memproses input bahan visual dan mengeluarkan resipi negeri Malaysia tradisional yang komprehensif dan mementingkan kesihatan. Pelaksanaan ini menunjukkan bahawa penggabungan pengecaman visual dengan enjin pengesyoran dipacu LLM dapat memperkasakan pengguna dengan berkesan untuk mengoptimumkan perbelanjaan makanan harian, meningkatkan penjejakan nutrisi, dan mengurangkan pembaziran makanan domestik dengan ketara melalui penggunaan bahan lebihan yang pintar.

TABLE OF CONTENTS

PAGE

LIST OF TABLES

PAGE

LIST OF FIGURES

PAGE

LIST OF ABBREVIATIONS

FYP	**-**	**Final Year Project**
AI	**-**	**Artificial Intelligence**
API	**-**	**Application Programming Interface**
BMI	**-**	**Body Mass Index**
CNN	**-**	**Convolutional Neural Network**
CRISP-DM	**-**	**Cross-Industry Standard Process for Data Mining**
CSS	**-**	**Cascading Style Sheets**
CV	**-**	**Computer Vision**
DOM	**-**	**Document Object Model**
DOSM	**-**	**Department of Statistics Malaysia**
ERD	**-**	**Entity-Relationship Diagram**
FYP	**-**	**Final Year Project**
HTML	**-**	**HyperText Markup Language**
HTTP	**-**	**Hypertext Transfer Protocol**
IDE	**-**	**Integrated Development Environment**
JPEG	**-**	**Joint Photographic Experts Group**
JS	**-**	**JavaScript**
JSON	**-**	**JavaScript Object Notation**
LLaMA	**-**	**Large Language Model Meta AI**
LLM	**-**	**Large Language Model**
NLP	**-**	**Natural Language Processing**
OpenDOSM	**-**	**Open Data Department of Statistics Malaysia**
PNG	**-**	**Portable Network Graphics**
PWA	**-**	**Progressive Web Application**

REST	**-**	**Representational State Transfer**
RM	**-**	**Ringgit Malaysia**
SDLC	**-**	**Software Development Life Cycle**
SHA	**-**	**Secure Hash Algorithm**
SPA	**-**	**Single Page Application**
SQL	**-**	**Structured Query Language**
TFLite	**-**	**TensorFlow Lite**
UI	**-**	**User Interface**
UX	**-**	**User Experience**

LIST OF ATTACHMENTS

CHAPTER 1: INTRODUCTION

1.1 Introduction

In contemporary society, rapid urbanization and fast-paced lifestyles have fundamentally altered daily dietary habits. Many individuals, particularly university students and young working adults increasingly rely on third-party prepared meals and commercial food delivery services. This shift is primarily driven by the perceived time-consuming nature of searching for appropriate recipes, systematically planning meals, and preparing raw ingredients (Anand et al., 2024). The rigorous demands of academic workloads and urban commuting leave this demographic with limited time and cognitive energy to dedicate to domestic culinary tasks, resulting in convenience frequently outweighing nutritional considerations. Consequently, this modern urban lifestyle contributes to the frequent consumption of highly processed, calorie-dense, and nutritionally imbalanced meals.

Simultaneously, poor pantry management exacerbates inefficiencies within domestic households. Consumers often procure grocery items without a structured, data-driven meal plan, leading to underutilized ingredients that perish before they can be effectively incorporated into meals (Anitha et al., 2023). Leftover vegetables, sauces, or partially used staple items are frequently discarded simply because individuals lack the creative culinary knowledge to repurpose them. While various digital recipe platforms currently exist in the market, the majority of these systems depend heavily on manual, text-based ingredient input by the user. This traditional approach is prone to human error, suffers from omission, and fails to accurately reflect real-time pantry availability, thereby reducing its practical viability for everyday use (Nor Azlan Shah et al., 2025). To mitigate these cascading issues, this project

introduces "The Fridge Chef," an AI-Powered Culinary Assistant that synergizes Convolutional Neural Networks (CNN) for visual ingredient recognition and Natural Language Processing (via Groq LLaMA-3) to dynamically generate budget-optimized, nutritionally balanced, and culturally appropriate meal plans.

1.2 Problem Statement

Despite the proliferation of digital lifestyle applications, significant gaps remain in effectively assisting low-to-middle-income demographics with holistic meal planning. The specific problems addressed in this research are:

i. Economic Constraints and Budgetary Limitations

University students and individuals managing strict financial constraints face significant socio-economic challenges in planning nutritionally adequate meals within strict financial limits (e.g., maintaining a daily food expenditure of under RM 10.00). Without the intervention of an intelligent budgeting assistant that tracks market prices, users are forced to either overspend their limited resources or compromise on nutritional quality to reduce daily costs.

ii. Household Food Waste and Inventory Inefficiency

A general lack of knowledge regarding ingredient substitution and leftover repurposing directly leads to the unnecessary disposal of edible food. The absence of an intelligent, automated system capable of recognizing existing pantry items and suggesting viable recipes aggravates domestic food waste and significantly diminishes overall cost efficiency (Boyd et al., 2024).

iii. Nutritional Tracking and Cultural Blind Spots

Many users lack access to real-time caloric estimation and nutritional feedback for home-cooked meals. Furthermore, mainstream culinary applications are predominantly Western-centric. They frequently fail to provide comprehensive filtering mechanisms for Halal dietary requirements or traditional Malaysian state

cuisines, creating a cultural gap that reduces the inclusivity and relevance of the application for local users (Chen & Chiang, 2025).

1.3 Objectives

To systematically resolve the aforementioned problem statements, this project embarks on the following core objectives:

i. To identify the system requirements for a multi-objective recommendation engine tailored for dynamic recipe generation.

ii. To design a multi-objective recommendation algorithm incorporating Natural Language Processing (LLM) to retrieve and format recipes constrained by a financial budget, caloric intake limits, and user cultural preferences (specifically Halal requirements and Traditional Malaysian State Cuisines).

iii. To develop a Computer Vision model utilizing Convolutional Neural Networks (CNN) to accurately recognize and classify local food ingredients from user-provided images.

iv. To implement and rigorously evaluate a mobile-web Progressive Web Application (PWA) prototype that processes ingredient images and interactively displays customized, health-conscious recipe solutions to the end-user.

1.4 Project Scope

To ensure the project remains focused, achievable, and aligned with the Bachelor of Computer Science curriculum, the scope is defined by the following parameters:

i. Target User Demographic:

The system is explicitly tailored for university students and young adults managing personal meal preparation under financial and temporal constraints, particularly those within the Budget-conscious households economic bracket.

ii. Dataset & Visual Recognition:

The computer vision pipeline will utilize a Convolutional Neural Network (CNN) trained specifically to recognize a predefined dataset of 22 common localized Malaysian food ingredients (eggs, tomatoes, onions, leafy greens, poultry) directly from the user's mobile camera.

iii. Algorithmic Focus & Integration:

The system architecture leverages the Groq LLaMA-3 API for NLP-driven recipe generation and caloric calculation. Financial calculations will be dynamically linked to the Malaysia OpenDOSM retail price database to ensure precise budgetary accuracy.

iv. Platform Deployment:

The frontend will be deployed as a Progressive Web Application (PWA) using HTML, CSS, and JavaScript, interfacing with a Python FastAPI backend to ensure lightweight processing efficiency and a native app-like experience without requiring a traditional App Store download.

1.5 Project Significance

The development of The Fridge Chef carries substantial socio-economic and environmental significance by addressing interconnected issues of financial constraints and poor pantry management. The project's creative integration of multiple AI technologies into a single Progressive Web Application (PWA) is what makes it highly significant. By automating ingredient recognition and providing creative solutions for leftover utilization, the system directly combats domestic food waste and promotes sustainable consumption behaviors.

Furthermore, the integration of real-time macroeconomic data via the OpenDOSM API adds an unparalleled degree of usefulness. For university students

and individuals managing strict financial constraints, this transforms the application into a vital financial tool. Rather than manually tracking grocery expenses, the system automatically aggregates the median retail prices of detected ingredients. This allows users to maximize their purchasing power and ensure that their meals are not only nutritionally balanced and Halal-compliant, but strictly within their daily budget.

From an academic and technological standpoint, the research shows how advanced artificial intelligence can be commoditized to solve every day, real-world problems. The combination of a Convolutional Neural Network (CNN) for visual recognition and a Large Language Model (LLaMA-3) for natural language recipe synthesis ensures that users of all technical backgrounds can easily adopt the system. Additionally, the platform is highly flexible and extensible for upcoming additions, such as grocery delivery integration or personalized diet-tracking dashboards.

1.6 Expected Output

The culmination of this research will deliver a fully functional software prototype of the AI-Powered Culinary Assistant deployed as a Progressive Web App (PWA). End-users will be able to capture a photograph of their refrigerator inventory, prompting the CNN and Groq pipeline to instantly identify the ingredients. The system will then output a highly curated, budget-optimized list of Halal-certified recipes. These recommendations will feature step-by-step cooking instructions, exact fractional meal costs based on live market data, and personalized macronutrient tracking calculated from the user's biometrics.

1.7 Report Organization

This report is systematically structured into six main chapters to provide a comprehensive overview of the project's development lifecycle.

1.7.1 Chapter 1: Introduction

This chapter provides the foundational background of the project, articulating the problem statements, defining the core objectives, establishing the project scope, and outlining the expected outcomes and significance of the proposed AI-driven culinary assistant.

1.7.2 Chapter 2: Literature Review and Project Methodology

Chapter 2 critically evaluates existing recipe recommendation systems and the application of deep learning in food recognition. It identifies the technological gaps the project aims to fill. The second half details the Agile Software Development Life Cycle (SDLC) employed to guide the system's engineering phases.

1.7.3 Chapter 3: Analysis

This chapter dissects the operational limitations of current manual culinary applications and thoroughly analyzes the proposed intelligent system. It includes the structural architecture of the system and a comprehensive work breakdown for the individual developer.

1.7.4 Chapter 4: Design

Chapter 4 presents the technical blueprint of the system. It elaborates on the intelligent system architecture, details the logical flowcharts for the Authentication, Computer Vision, and Generative modules, and provides the Entity-Relationship Diagram (ERD) defining the system's database structure.

1.8 Summary

Chapter 1 has established the fundamental premise of "The Fridge Chef." By identifying the critical issues of domestic food waste and strict financial constraints among the low-to-middle income demographics and cost-conscious users, this chapter outlined the necessity for an automated, AI-driven culinary assistant. The objectives, scope, and expected outcomes were clearly defined to set a focused trajectory for the research. The subsequent chapter will delve into the existing literature and detail the Agile methodology utilized to bring this intelligent system to fruition.

CHAPTER 2: LITERATURE REVIEW AND PROJECT METHODOLOGY

2.1 Introduction

This chapter provides a comprehensive overview of the background literature, domain knowledge, existing systems, and techniques relevant to the development of "The Fridge Chef" an AI-Powered Culinary Assistant. The primary objective is to critically examine the underlying principles of computer vision (CV) for ingredient recognition, natural language processing (NLP) for dynamic recipe generation, and economic optimization within dietary planning. By exploring current advancements and identifying the limitations of existing solutions, this chapter establishes a theoretical foundation that informs the architectural and methodological choices made in designing the system. Furthermore, this chapter outlines the Agile Software Development Life Cycle (SDLC) and CRISP-DM methodologies adopted for building, training, and deploying the application, justifying the technological approaches taken to ensure the system effectively addresses the defined problem statements.

2.2 Facts and Findings

2.2.1 Domain

The operational domain of this project resides at the intersection of Applied Artificial Intelligence (Computer Vision and NLP), Health Informatics, and Behavioral Economics. The multidisciplinary nature of The Fridge Chef enables it to transcend traditional recipe applications by offering intelligent, real-time pantry management and financially constrained meal planning. The relevant subdomains integrated into this system are detailed below:

2.2.1.1 Image-Based Food Recognition

Accurate food and ingredient recognition remains a complex challenge in computer vision due to the high intra-class variance and low inter-class variance of food items (e.g., differentiating between a potato and a yellow onion under varying lighting conditions). Traditional manual input systems are prone to friction and user abandonment. Recent studies highlight the efficacy of Deep Learning, specifically Convolutional Neural Networks (CNNs), in automating this process. Pan et al. (2017) demonstrated the viability of CNN architectures like DeepFood for multi-class ingredient classification, achieving significant accuracy improvements over traditional hand-crafted feature extraction methods. Similarly, Ciocca et al. (2020) emphasized the importance of deep features in recognizing the state of food images, a critical aspect when evaluating pantry inventory. The Fridge Chef builds upon these foundations by training a localized CNN model to specifically identify 22 common Malaysian food ingredients, bridging the gap between generic global datasets and regional culinary realities.

Furthermore, while local CNNs provide offline efficiency and privacy, they face limitations when scanning complex, highly cluttered scenes like a fully stocked refrigerator. To address this, modern systems are increasingly adopting Multi-Modal Large Language Models (such as the Gemini Vision API) which utilize advanced Transformer architectures to map complex spatial relationships, allowing for the simultaneous extraction of multiple overlapping ingredients from a single image.

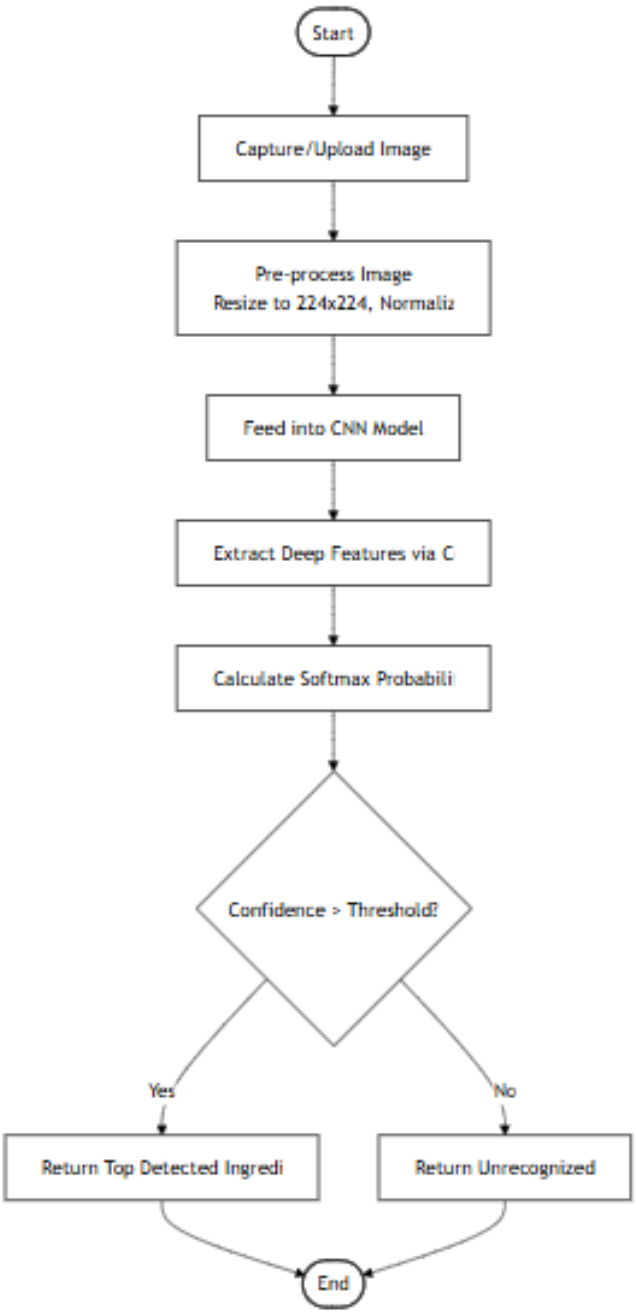


Figure 2.1: Flowchart of CNN-Based Food Scanner

Figure 2.1 shows the flowchart for the CNN-based food scanner. It details the step-by-step process of how an uploaded image is resized, passed into the model, and classified to output an ingredient name.

```python
@app.post("/api/scan/cnn")
async def scan_cnn(file: UploadFile = File(...)):
    if interpreter is None:
        raise HTTPException(status_code=503, detail="CNN model not available.")

    contents = await file.read()
    image = Image.open(io.BytesIO(contents)).convert("RGB")

    # Preprocess and feed into TFLite Interpreter
    input_data = preprocess_image(image)
    interpreter.set_tensor(input_details[0]['index'], input_data)
    interpreter.invoke()

    # Extract Softmax Probabilities
    output_data = interpreter.get_tensor(output_details[0]['index'])
    idx = int(np.argmax(output_data[0]))
    confidence = float(output_data[0][idx])
    name = CLASS_NAMES[idx]

    return {"items": [name], "confidence": confidence}
```

Figure 2.2: CNN Image Classification

The image recognition functionality is deployed via the /api/scan/cnn endpoint using FastAPI. Upon receiving a multipart image upload from the user, the image is immediately decoded from a byte stream and converted to RGB format. It is then resized to 224x224 and normalized through the preprocess_image() function to match the exact input shape expected by the model. The input tensor is fed into the optimized TensorFlow Lite (TFLite) interpreter, which invokes the pre-trained Convolutional Neural Network. The system extracts the probabilities from the Softmax output layer, using argmax to determine the highest confidence index, which is then mapped to the predefined array of 22 Malaysian ingredient class names.

2.2.1.2 Conversational AI and NLP in Culinary Assistance

While static databases retrieve recipes based on keyword matching, they lack the contextual nuance required to synthesize new cooking protocols based on strict user constraints. The integration of Large Language Models (LLMs) has revolutionized personalized dietary recommendations. Khamesian et al. (2025) note that LLM-based generators significantly enhance nutritional adherence by dynamically adapting to user preferences. The Fridge Chef employs the LLaMA-3 model via the Groq API to function as a conversational culinary assistant.

Crucially, generating massive structural payloads (such as step-by-step JSON recipes) via traditional GPU clusters incurs heavy latency, negatively impacting the user experience. To overcome this, The Fridge Chef utilizes Groq's Language Processing Unit (LPU) architecture, a specialized inference engine that processes language deterministically. This ensures that the NLP pipeline synthesizes step-by-step recipes constrained by Halal dietary laws, caloric limits, and the exact ingredients identified by the CV module in a matter of milliseconds, providing true real-time personalization.

```python
# Requesting a highly structured JSON recipe from Groq
try:
    response = groq_client.chat.completions.create(
        model="llama-3.3-70b-versatile",
        messages=[
            {"role": "system", "content": "You are an API that outputs ONLY valid JSON. No markdown."},
            {"role": "user", "content": prompt}
        ],
        response_format={"type": "json_object"}
    )
    data = json.loads(response.choices[0].message.content)
    return {"ideas": data.get("ideas", [])}
except Exception as e:
    raise HTTPException(status_code=500, detail=str(e))
```

Figure 2.3: JSON Meal Plan Prompt for Groq API

To generate personalized cooking protocols, The Fridge Chef integrates with the Groq API to utilize the LLaMA-3 Large Language Model. Because the Progressive Web App requires strictly formatted data to render interactive UI components, the system employs strict Prompt Engineering. By defining the LLM's role as a "JSON-only API" in the system message and enforcing response_format= {"type": "json_object"}, the backend ensures the model does not return conversational conversational markdown. The resulting response is parsed directly into a Python dictionary, allowing the system to flawlessly render complex recipe arrays on the frontend.

2.2.1.3 Economic Optimization in Diet

A critical yet underexplored subdomain in nutritional informatics is the integration of real-time macroeconomic data. University students and budget-conscious individuals often operate under strict daily budgets (< RM10). While systems like MyFitnessPal address caloric tracking, they ignore the financial cost of

meal preparation. The Fridge Chef introduces a novel economic dimension by integrating real-time median retail prices from the Malaysia Open Data Portal (OpenDOSM). This deterministic integration ensures that meal plans are not only nutritionally balanced but mathematically proven to fit within the user's defined financial budget, directly addressing food inflation and economic hardship.

```
# Loading Median Retail Prices from Malaysia Open Data Portal
def load_price_database():
    items = {}
    with open(LOOKUP_FILE, encoding='utf-8') as f:
        for row in csv.DictReader(f):
            code = row.get('item_code', '').strip()
            name = row.get('item', '').strip()
            if code and name:
                items[code] = {
                    'name_malay': name,
                    'unit':       row.get('unit', '').strip(),
                    'category':   row.get('item_category', '').strip(),
                    'group':      row.get('item_group', '').strip(),
                }
    return items
```

Figure 2.4: OpenDOSM Price Aggregation Logic

To prevent budgetary hallucinations by the AI, financial constraints are handled by a deterministic macroeconomic pipeline linked to the Malaysia Open Data Portal (OpenDOSM). The load_price_database() function reads and parses localized item data, mapping unique item codes to their respective categories and measurement units. This structural mapping enables the system to cross-reference the ingredients detected by the CNN with actual market retail prices, ensuring that the total meal cost calculated by the application remains strictly within the user's RM (Ringgit Malaysia) budget limits.

2.2.1.4 Authentication and User Management

Secure user authentication is a foundational requirement for any personalized culinary application. To provide highly customized diet plans based on personal biometrics (e.g., weight, height, caloric limits) and strict dietary restrictions (e.g., Halal certification, allergies), the system must securely maintain persistent user profiles. Without a robust user management domain, applications cannot effectively track historical data such as longitudinal cost savings or recurring pantry inventories.

Modern web applications typically implement authentication via cryptographic hashing protocols (such as SHA-256 or bcrypt) to ensure password security, coupled with session management to maintain state across HTTP requests. The Fridge Chef adopts these domain standards by implementing a secure login and registration module within the FastAPI backend. This ensures that personal financial budgets and dietary constraints are securely siloed and instantly retrievable upon successful authentication, providing a seamless and highly personalized user experience.

```python
@app.post("/api/login")
def login(req: LoginRequest):
    # Validate against securely hashed credentials in SQLite
    user = db.login_user(req.username, req.password)
    if not user:
        raise HTTPException(status_code=401, detail="Incorrect username or password.")

    # Return session data and biometric constraints
    return {
        "success": True,
        "user_id": user["id"],
        "username": user["username"],
        "bmi": user.get("bmi", 22.5),
        "health_goal": user.get("health_goal", "Maintain Current Weight"),
        "allergies": user.get("allergies", "")
    }
```

Figure 2.5: User Authentication and Validation

The authentication flow is handled via the /api/login endpoint. The function receives the user's credentials in a structured JSON payload and passes them to the database module, where the plaintext password is mathematically verified against the stored cryptographic hash (SHA-256). Upon successful validation, rather than immediately dropping the connection, the backend compiles and returns the user's critical biometric data (such as their calculated BMI, health goals, and allergies). This allows the frontend Progressive Web App to instantly cache these constraints into local storage and personalize all subsequent culinary recommendations without requiring repeated database queries.

2.2.2 Existing System

A variety of digital nutrition and recipe recommendation systems exist in the commercial market and academic literature. However, an analysis of these systems reveals a persistent gap in combining visual recognition, real-time budgeting, and

localized cultural constraints. Table 2.1 compares several prominent systems against the proposed application.

2.2.2.1 Hardware Used by Existing Systems

Current recipe and dietary management systems primarily rely on standard consumer hardware. Applications such as SuperCook and MyFitnessPal are designed to operate on ubiquitous mobile devices (smartphones and tablets) equipped with standard RGB cameras for barcode scanning. Conversely, more advanced academic prototypes utilizing heavy object detection models (such as YOLOv8) often require significant computational hardware, including dedicated cloud GPUs (NVIDIA RTX series) or high-end mobile processors to handle real-time video feeds without severe latency.

2.2.2.2 Software Used by Existing Systems

From a software perspective, existing applications employ a wide variety of stacks. Commercial systems like MyFitnessPal rely on static, centralized relational databases paired with native mobile applications (iOS/Android) built using frameworks like Swift or Kotlin. Web-based keyword matchers like SuperCook utilize lightweight frontend web technologies (HTML/JS) connected to basic search algorithms. More experimental AI-driven systems integrate deep learning frameworks such as PyTorch or TensorFlow, deploying Python-based backend servers (e.g., Flask or Django) to handle the complex image processing and object clustering algorithms before returning results to the user interface.

Table 2.1: Comparison of Existing Recipe and Nutrition Systems

System Name	Core Features	Weaknesses	Technology Used
SuperCook (Opezlo, 2024)	Recipe matching based on user-input ingredients.	Requires manual data entry, no real-time budget tracking, lacks localized Asian/Halal filters.	Database keyword matching, Web App.

MyFitnessPal (MyFitnessPal Inc., 2024)	Calorie tracking, macro analysis, barcode scanning.	Highly Western-centric database, lacks predictive recipe generation from leftover ingredients, no cost estimation.	Mobile App, Static Database.
PlantJammer (Plant Jammer A/S, 2024)	AI-driven ingredient substitution and recipe generation.	Exclusively vegetarian, no image recognition capabilities, premium features behind a paywall.	AI recommendation engine, Mobile App.
YOLO-based Recipe Recommender (Swain et al., 2023)	Object detection for ingredients and subsequent recipe suggestions.	Computationally heavy for mobile-web deployment; lacks financial budgeting and cultural personalization.	YOLO Object Detection, Clustering.
The Fridge Chef (Proposed)	Real-time CNN ingredient scanning, LLaMA-3 recipe synthesis, OpenDOSM budget tracking, Halal/State cuisine filtering.	Requires stable internet connection for LLM processing and live price fetching.	Python FastAPI, PWA, CNN, LLaMA-3 (Groq), OpenDOSM API.

As evidenced by Table 2.1, while existing platforms excel in isolated tasks (calorie tracking or keyword recipe matching), they fail to offer a cohesive, automated experience that factors in the financial constraints of low-to-middle-income users. The

Fridge Chef strategically resolves these deficiencies by merging computer vision for frictionless input with deterministic financial logic.

2.2.3 Technique

The Fridge Chef employs a hybrid architecture, combining the following core AI and software engineering techniques:

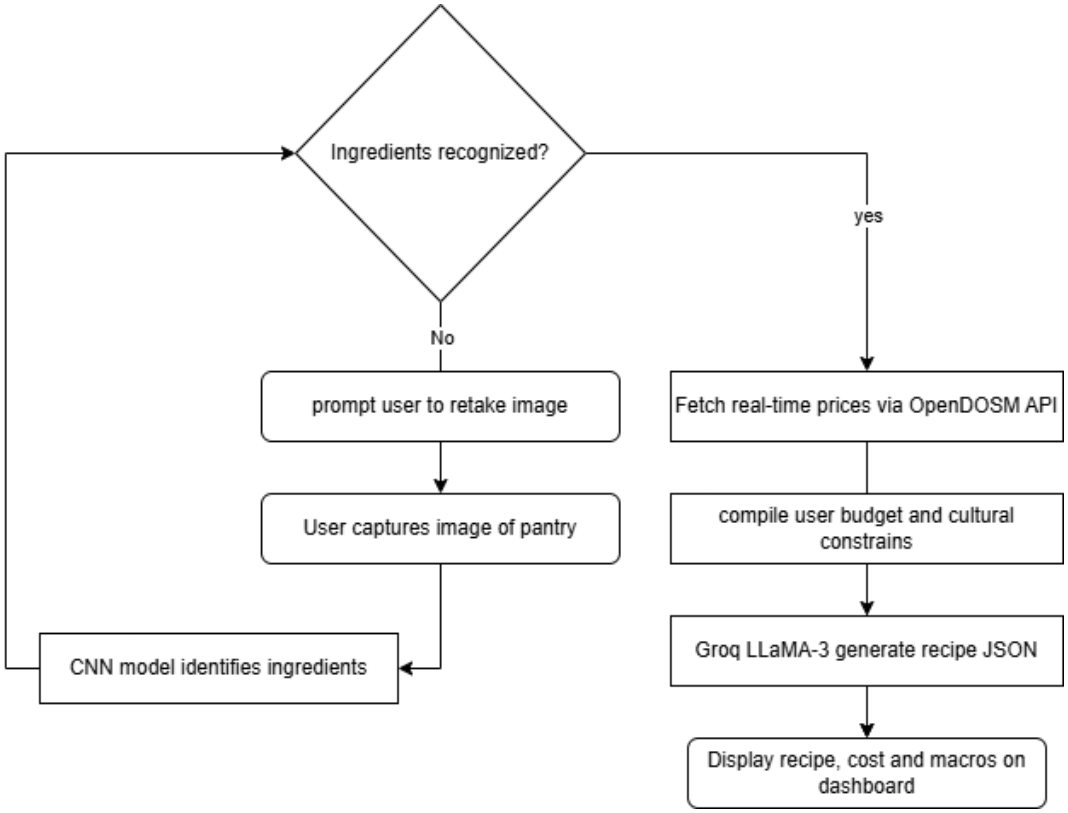


Figure 2.6: Flowchart showing the high-level logic of how user input passes through the CNN and NLP to generate a recipe

Figure 2.6 shows the overall logic of The Fridge Chef. It illustrates how the system processes user inputs, integrates with the CNN model and OpenDOSM API, and generates a final recipe using the LLaMA-3 AI model.

Table 2.2: Techniques Used and Considered

Technique	Description	Applied In The Fridge Chef	Reason For Inclusion/Exclusion

Convolutional Neural Networks (CNN)	Deep learning architecture designed to process grid-like topology (images) through convolutional filters.	Yes	Selected for ingredient classification due to its superior feature extraction capabilities and lightweight deployment potential compared to heavier bounding-box models like YOLO.
YOLO (You Only Look Once)	Real-time object detection algorithm that identifies objects and their bounding boxes.	No	Excluded due to the computational overhead it introduces for mobile-web platforms. A pure CNN classification model is sufficient and faster for identifying an array of ingredients in a single frame.
Large Language Models (LLaMA-3)	Advanced NLP models capable of zero-shot reasoning and contextual text generation.	Yes	Utilized via Groq API to synthesize dynamic, culturally relevant cooking instructions and handle complex multi-variable constraints (Halal, calories, budget).

Rule-Based Decision Trees	Deterministic logic pathways using predefined IF-THEN rules.	Partial	Used exclusively for the financial calculation module (OpenDOSM price aggregation) to prevent LLM arithmetic hallucinations and ensure absolute mathematical accuracy.

Table 2.3: Comparison of AI Vision Techniques for Food Recognition

AI Technique	Primary Mechanism	Processing Speed	Cloud Dependency
Standard CNN (e.g., MobileNet, VGG)	Image classification via convolutional feature extraction.	Extremely Fast	Offline capable (TFLite)
YOLO (You Only Look Once)	Single-shot bounding box regression and classification.	Fast (Hardware dependent)	Offline capable (Heavy)
Multi-Modal Transformers (e.g., Gemini)	Encodes full images into text using self-attention mechanisms.	Moderate (Latency)	Fully Cloud-dependent

2.2.3.1 Comparison with Recent Academic Research

While commercial applications offer baseline dietary tracking, recent academic literature has made significant strides in employing Artificial Intelligence for culinary assistance. However, a critical analysis of current academic works reveals a persistent gap in addressing real-time economic constraints. Table 2.2 below compares recent academic frameworks against the proposed system.

Table 2.4: Comparison of Proposed System with Recent Academic Research

Research Paper	**Core AI Methodology**	**Strengths**	**Limitations**
Swain et al. (2023)	YOLO-based object detection with clustering algorithms.	High accuracy in multi-object bounding box detection.	Computationally heavy; lacks integration of real-world financial budgeting.
Anitha et al. (2023)	CNN-based image classification for recipe generation.	Efficient feature extraction for standard ingredient recognition.	Employs static recipe retrieval rather than dynamic, LLM-driven synthesis.
Nor Azlan Shah et al. (2025)	CNN Integrated Mobile Application for food recognition.	Focuses on mobile deployment and user accessibility.	Does not address cultural filtering (Halal) or dynamic macroeconomic (cost) variables.
The Fridge Chef (Proposed)	Hybrid: CNN (Vision) + LLaMA-3 (NLP) + Deterministic Logic.	Real-time visual inference combined with zero-shot NLP reasoning.	Fills the academic gap by seamlessly combining AI recognition with deterministic, real-time economic (OpenDOSM) and

			cultural constraints.

As highlighted in Table 2.2, while academic precedents successfully utilize CNNs and YOLO architectures for visual recognition, they operate in an economic vacuum. *The Fridge Chef* uniquely advances the domain by introducing a multi-objective recommendation engine that mathematically guarantees budget compliance while synthesizing culturally relevant (Halal) meals.

2.2.3.2 Convolutional Neural Networks (CNN) for Image Classification

A Convolutional Neural Network (CNN) is a class of deep feed-forward artificial neural networks primarily used to analyze visual imagery. Unlike traditional algorithms that require manual feature extraction, CNNs use convolutional layers to automatically learn spatial hierarchies of features from input images such as edges, textures, and complex shapes. By employing pooling layers to reduce spatial dimensionality and dense layers for final classification, CNNs are highly effective at multi-class object recognition. The Fridge Chef employs this technique specifically because it excels at classifying localized food ingredients despite high intra-class variance (recognizing different shapes and lighting conditions of an onion) while remaining lightweight enough to be deployed via TensorFlow Lite.

2.2.3.3 Large Language Models (LLM) for Natural Language Generation

Large Language Models, such as LLaMA-3, represent a significant advancement in Natural Language Processing (NLP). These generative pre-trained transformers process input text (prompts) through billions of parameters and multiple attention heads to predict and synthesize highly contextual, human-like text responses. In the context of culinary assistance, traditional rule-based algorithms are too rigid to handle complex, multi-variable constraints (combining 5 random ingredients while ensuring the recipe is Halal and strictly under 450 calories). By utilizing the LLaMA-3 technique via the Groq API, the system gains the cognitive flexibility required to dynamically generate personalized cooking protocols that rule-based systems cannot provide.

2.2.3.4 Deterministic Algorithms for Financial Aggregation

While Generative AI is powerful for synthesizing language, it suffers from "arithmetic hallucination," meaning LLMs are notoriously unreliable at performing exact mathematical operations or fetching real-time market data. To resolve this, a deterministic algorithmic technique is employed alongside the AI. Deterministic algorithms follow a strict, predictable sequence of rules if a specific input is given, the exact same output will always be produced. By linking a deterministic Python pipeline to the Malaysia Open Data Portal (OpenDOSM), the system ensures that the financial calculations for the meal plan are 100% mathematically accurate, completely bypassing the LLM's unreliability with numbers.

2.2.4 Dataset and Model Training

The success of the Computer Vision module relies heavily on the quality and contextual relevance of its training data. Unlike generic food datasets (e.g., Food-101), which predominantly feature Western dishes, The Fridge Chef requires a localized approach.

2.2.4.1 Dataset Overview

The model is trained on a custom dataset comprising 22 common Malaysia food ingredients, including staple items such as Bawang Merah (Red Onion), Cili Padi (Bird's Eye Chili), *Brokoli* (Broccoli), *Lobak merah* (Carrot), and *Daging* (Beef). This localized dataset ensures the model's high applicability in typical Malaysian domestic kitchens.

Table 2.5: List of Localized Ingredients in Training Dataset

Category	**Ingredient Classes (Model Labels)**	**Total Classes**
Proteins	Beef	1
Vegetables & Fruits	Bean, Bitter Gourd, Bottle Gourd, Brinjal, Broccoli, Cabbage, Carrot, Cauliflower,	16

	Cucumber, Eggplant, Papaya, Potato, Pumpkin, Radish, Tomato, Capsicum	
Aromatics & Spices	Garlic, Ginger, Lemongrass, Onion, Red Chili,	5
	Total Ingredient Classes:	22

The dataset was curated to ensure the CNN model is specifically tailored to recognize staple items commonly found in Malaysian domestic kitchens and budget-conscious individuals.

Image Preprocessing and Augmentation

To enhance the robustness of the CNN model against varying real-world conditions (poor kitchen lighting, different camera angles, and background clutter), the dataset undergoes rigorous preprocessing. Techniques applied include:

- **Resizing and Normalization:** Scaling all images to a uniform pixel dimension (224x224) and normalizing pixel values to a [0, 1] range to accelerate convergence.

- **Data Augmentation:** Applying horizontal flipping, random rotation, zooming, and brightness shifting to artificially expand the dataset and mitigate overfitting (Boyd et al., 2024).

```
data_augmentation = keras.Sequential(
  [
    layers.RandomFlip("horizontal", input_shape=(img_height, img_width, 3)),
    layers.RandomRotation(0.1),
    layers.RandomZoom(0.1),
  ]
)
```

Figure 2.7 data_augmentation

2.2.4.2 Model Architecture

The CNN architecture is developed using the TensorFlow/Keras framework in Python. It features multiple convolutional layers for feature extraction, followed by max-pooling layers to reduce spatial dimensions, and dense layers utilizing softmax activation to output probabilities across the 22 ingredient classes. The final model is optimized for rapid inference, allowing seamless integration with the FastAPI backend.

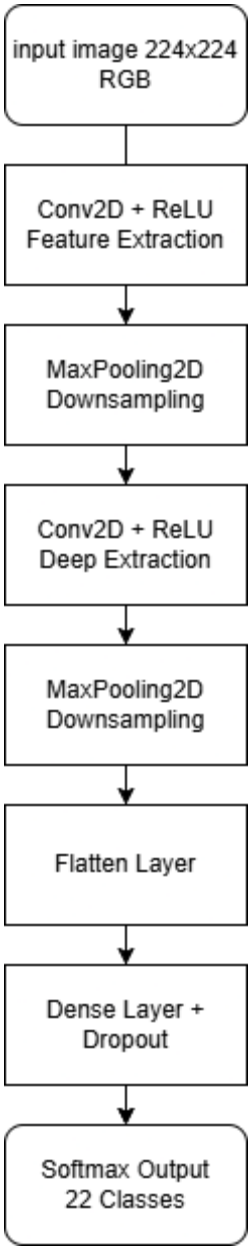


Figure 2.8: Diagram of CNN Architecture

Figure 2.7 presents the structural architecture of the Convolutional Neural Network (CNN). It details the sequence of convolutional layers for feature extraction, pooling layers for downsampling, and dense layers for final classification.

2.2.4.3 Model Training Process

The Convolutional Neural Network (CNN) was trained to recognize the 22 localized ingredient classes using the Keras framework. The training process was optimized using the Adaptive Moment Estimation (Adam) optimizer, selected for its computational efficiency and low memory requirements. The loss function employed was Sparse Categorical Crossentropy, which is ideal for multi-class classification tasks where the labels are provided as integers rather than one-hot encoded vectors.

The model was trained over 15 epochs with a batch size of 32. Throughout the training phase, both training and validation accuracy and loss were continuously monitored to detect potential overfitting. The integration of dropout layers in the architecture ensured that the model generalized well to unseen data, maintaining high validation accuracy.

```
epochs = 15
history = model.fit(
  train_ds,
  validation_data=val_ds,
  epochs=epochs
)
```

Figure 2.9: CNN Model Training Execution

Figure 2.9 displays the Keras model.fit() function initiating the training sequence. The dataset is passed into the model along with the validation data, allowing the neural network to learn the localized ingredient features over 15 complete epochs.

```
Epoch 1/15
277/277 ━━━━━━━━━━━━━━━━━━━━ 120s 420ms/step - accuracy: 0.9183 - loss: 0.3190 - val_accuracy: 0.5392 - val_loss: 5.0990
Epoch 2/15
277/277 ━━━━━━━━━━━━━━━━━━━━ 113s 407ms/step - accuracy: 0.9791 - loss: 0.0747 - val_accuracy: 0.5442 - val_loss: 5.9413
Epoch 3/15
277/277 ━━━━━━━━━━━━━━━━━━━━ 115s 414ms/step - accuracy: 0.9860 - loss: 0.0489 - val_accuracy: 0.5447 - val_loss: 6.5862
Epoch 4/15
277/277 ━━━━━━━━━━━━━━━━━━━━ 114s 412ms/step - accuracy: 0.9905 - loss: 0.0308 - val_accuracy: 0.5457 - val_loss: 7.0943
Epoch 5/15
277/277 ━━━━━━━━━━━━━━━━━━━━ 116s 420ms/step - accuracy: 0.9908 - loss: 0.0305 - val_accuracy: 0.5452 - val_loss: 7.2835
Epoch 6/15
277/277 ━━━━━━━━━━━━━━━━━━━━ 114s 411ms/step - accuracy: 0.9907 - loss: 0.0304 - val_accuracy: 0.5452 - val_loss: 7.5652
Epoch 7/15
277/277 ━━━━━━━━━━━━━━━━━━━━ 113s 409ms/step - accuracy: 0.9922 - loss: 0.0229 - val_accuracy: 0.5462 - val_loss: 8.4976
Epoch 8/15
277/277 ━━━━━━━━━━━━━━━━━━━━ 114s 410ms/step - accuracy: 0.9946 - loss: 0.0169 - val_accuracy: 0.5447 - val_loss: 8.6109
Epoch 9/15
277/277 ━━━━━━━━━━━━━━━━━━━━ 114s 411ms/step - accuracy: 0.9915 - loss: 0.0258 - val_accuracy: 0.5442 - val_loss: 8.7391
Epoch 10/15
277/277 ━━━━━━━━━━━━━━━━━━━━ 113s 408ms/step - accuracy: 0.9939 - loss: 0.0178 - val_accuracy: 0.5452 - val_loss: 9.6165
Epoch 11/15
277/277 ━━━━━━━━━━━━━━━━━━━━ 114s 412ms/step - accuracy: 0.9952 - loss: 0.0136 - val_accuracy: 0.5462 - val_loss: 9.8984
Epoch 12/15
277/277 ━━━━━━━━━━━━━━━━━━━━ 113s 408ms/step - accuracy: 0.9950 - loss: 0.0162 - val_accuracy: 0.5457 - val_loss: 10.3171
Epoch 13/15
...
Epoch 14/15
277/277 ━━━━━━━━━━━━━━━━━━━━ 117s 422ms/step - accuracy: 0.9937 - loss: 0.0185 - val_accuracy: 0.5462 - val_loss: 10.0998
Epoch 15/15
277/277 ━━━━━━━━━━━━━━━━━━━━ 120s 434ms/step - accuracy: 0.9942 - loss: 0.0184 - val_accuracy: 0.5452 - val_loss: 10.7094
```

Figure 2.10: (The epoch output logs)

Figure 2.10 illustrates the terminal logs during the model training phase. As the epochs progressed, the model successfully minimized its loss function and achieved an exceptionally high training accuracy of over 99.4%, demonstrating its strong capacity to recognize the 22 Malaysian ingredients.

2.2.4.4 Model Conversion to TensorFlow Lite (TFLite)

While deep learning models trained in TensorFlow are highly accurate, they are often computationally heavy and resource-intensive, making them unsuitable for direct integration into lightweight web backends. To resolve this, the fully trained CNN model was converted into the TensorFlow Lite (.tflite) format.

This conversion process significantly compresses the model's footprint by applying post-training quantization, which reduces the precision of the model's weights without drastically sacrificing inference accuracy. The resulting culinary_assistant_model.tflite file is highly optimized for rapid, real-time inference on the FastAPI backend, allowing the system to instantly classify user-uploaded images with minimal latency.

```python
# 1. Create a new model that skips the data_augmentation layer
export_model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(img_height, img_width, 3))
])
for layer in model.layers[1:]:
    export_model.add(layer)

# 2. Use the new EXPORT function (Required for TensorFlow 2.16+)
export_model.export('my_saved_model')

# 3. Convert it
converter = tf.lite.TFLiteConverter.from_saved_model('my_saved_model')
tflite_model = converter.convert()

# 4. Save to TFLite
with open('culinary_assistant_model.tflite', 'wb') as f:
  f.write(tflite_model)

print("Model successfully exported to culinary_assistant_model.tflite!")
```

Figure 2.11: The TFLite Converter Proccess

Figure 2.11 show how model that convert into to TFLite file.

2.3 Project Methodology

The development of The Fridge Chef adopts a hybrid methodological approach to accommodate both the software engineering demands of a Progressive Web Application (PWA) and the data science requirements of training a Convolutional Neural Network. Specifically, the project integrates the Agile Software Development Life Cycle (SDLC) with the Cross-Industry Standard Process for Data Mining (CRISP-DM).

2.3.1 Agile Software Development Life Cycle (SDLC)

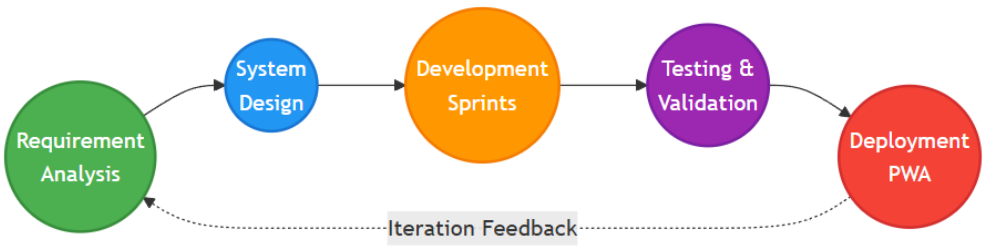


Figure 2.12: Diagram showing the Agile SDLC Phases (Requirement Analysis, Design, Sprints, Testing, Deployment)

Figure 2.8 displays the Agile Software Development Life Cycle (SDLC) utilized for this project. It highlights the continuous, iterative process of requirement analysis, design, development, testing, and deployment.

Agile methodology was selected for the application's frontend and backend development due to its iterative nature and adaptability to changing requirements.

- **Requirement Analysis:** Defining constraints such as OpenDOSM integration and LLM prompt engineering.

- **Design:** Architecting the FastAPI backend, Groq LLM pipelines, and the PWA user interface.

- **Development (Sprints):** Developing the system in iterative sprints, allowing for continuous refinement of the user interface and API integrations.

- **Testing:** Continuous unit testing of the OpenDOSM mathematical logic to prevent budget hallucinations.

- **Deployment:** Releasing the application as a PWA, ensuring cross-platform accessibility without the overhead of native app store deployment.

2.3.2 CRISP-DM for AI Integration

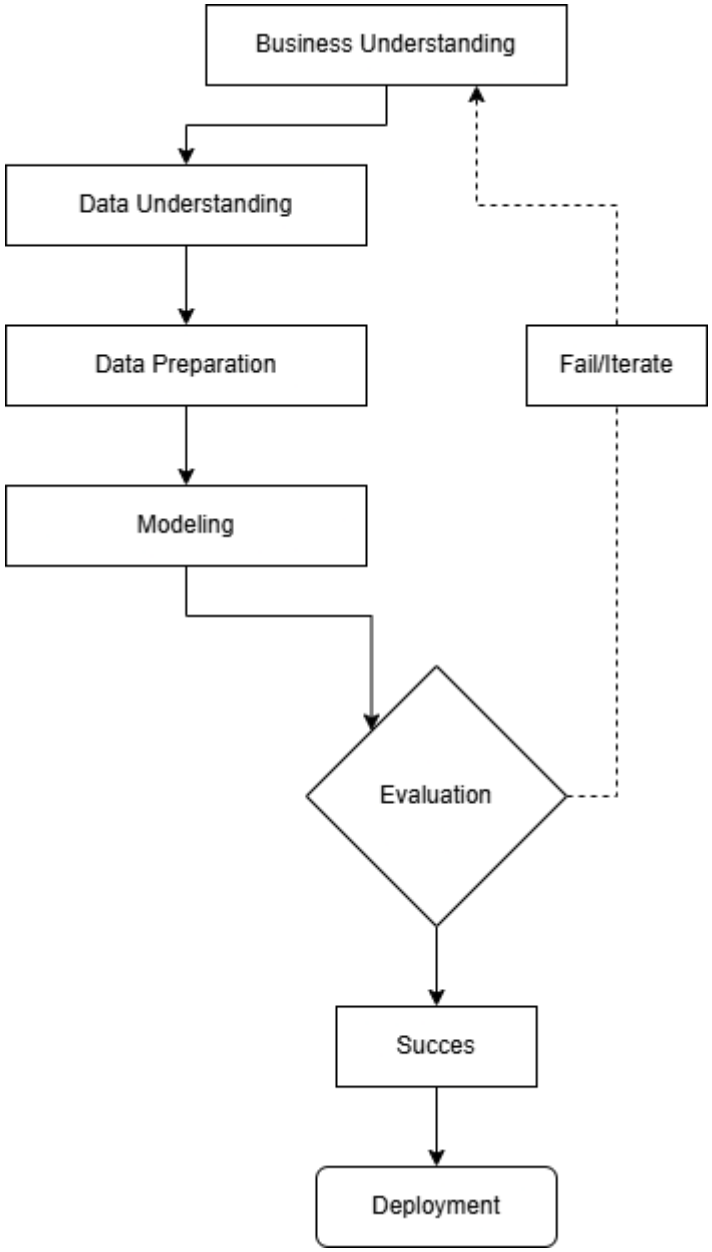


Figure 2.13: Diagram showing the CRISP-DM cycle (Business Understanding, Data Understanding, Preparation, Modeling, Evaluation, Deployment)

Figure 2.9 illustrates the CRISP-DM methodology used for the AI integration phase. It outlines the structured flow from business understanding down to final model evaluation and deployment.

To structure the development of the CNN ingredient recognition model, the CRISP-DM framework was utilized:

- **Business Understanding:** Recognizing the need to reduce food waste and optimize budgets using automated visual input.

- **Data Understanding & Preparation:** Collecting and preprocessing the 22-class Malaysian ingredient dataset (resizing, augmentation, and normalization).

- **Modeling:** Training the CNN using TensorFlow/Keras and optimizing hyperparameters to maximize prediction accuracy.

- **Evaluation:** Validating the model against a test set of localized ingredients, analyzing confusion matrices to identify misclassifications.

- **Deployment:** Exporting the trained model and integrating it into the FastAPI backend for real-time inference upon user image uploads.

2.3.3 Experimental Parameters and Measurements

Evaluating the performance of the AI models is critical to ensuring the reliability of The Fridge Chef. The CNN ingredient classification model is primarily measured using standard classification metrics:

- **Accuracy:** To determine the overall percentage of correctly identified ingredients from the test dataset.

- **Loss:** Monitored via the Sparse Categorical Crossentropy function to measure the disparity between the predicted probabilities and the true labels.

For the Natural Language Processing (NLP) component, the LLaMA-3 model's performance is qualitatively measured based on its ability to strictly adhere to user constraints (ensuring zero hallucinations regarding Halal requirements and strictly maintaining the budget limits fetched from OpenDOSM).

2.3.4 Justification of Methodology

The integration of the Agile Software Development Life Cycle (SDLC) with the Cross-Industry Standard Process for Data Mining (CRISP-DM) was chosen because it perfectly aligns with the hybrid nature of The Fridge Chef.

CRISP-DM provided a structured, rigorous framework necessary for the data-intensive tasks of collecting, augmenting, and training the Malaysian ingredient image dataset. Conversely, Agile SDLC was essential for the rapid prototyping and iterative development of the Progressive Web Application (PWA) and backend API. This dual-methodology approach ensured that the complex AI modeling did not bottleneck the software engineering process, allowing both the frontend interfaces and the machine learning models to be developed, tested, and deployed synchronously.

2.4 Project Requirements

2.4.1 Software Requirements

- Python & FastAPI: For robust backend development and routing.
- TensorFlow/Keras: For building and training the CNN food recognition model.
- Groq API (LLaMA-3): For high-speed, NLP-driven recipe generation.
- OpenDOSM API: For fetching live median retail prices of ingredients in Malaysia.
- HTML, CSS, JavaScript: For frontend PWA development

2.4.2 Hardware Requirements

- Laptop/PC with GPU: Required for efficient CNN model training and testing (e.g., NVIDIA RTX series).
- Smartphone with Camera: For testing the PWA interface and capturing live ingredient images.

2.5 Project Schedule and Milestones

A Gantt chart was utilized to manage the development timeline across a standard 14-week Final Year Project schedule. Key milestones include:

1. **Weeks 1-3:** Requirement analysis, literature review, and dataset collection.
2. **Weeks 4-6:** CNN model training, augmentation, and validation (CRISP-DM).
3. **Weeks 7-9:** Backend API development, integrating Groq LLaMA-3 and OpenDOSM.
4. **Weeks 10-12:** PWA frontend development, system integration, and UI/UX refinement.
5. **Weeks 13-14:** Comprehensive system testing, debugging, and final report documentation.

2.6 Summary

This chapter critically reviewed the domains of image-based food recognition, conversational culinary AI, and economic optimization within dietary tracking. By comparing existing applications such as SuperCook, MyFitnessPal, and YOLO-based systems, a clear gap was identified: the lack of a cohesive platform that merges frictionless visual input with deterministic financial logic for low-income demographics. The Fridge Chef addresses this by combining a custom-trained CNN for localized Malaysian ingredients with the Groq LLaMA-3 model and OpenDOSM price data. The hybrid Agile and CRISP-DM methodologies adopted for this project provide a structured yet flexible framework, ensuring the delivery of a robust, highly personalized, and mathematically accurate culinary assistant.

CHAPTER 3: REQUIREMENT ANALYSIS

3.1 Introduction

Understanding the system's core needs, constraints, and user expectations depends heavily on the analysis phase. It entails a methodical evaluation of the issues found in the literature study in order to convert them into precise, workable system requirements. This stage of "The Fridge Chef" project is to evaluate the shortcomings of current recipe and pantry management systems, determine the requirements of users looking for budget-friendly culinary advice, and provide a set of features that the system should have. This chapter explains how The Fridge Chef fills the gaps found in current procedures using structured modeling tools such as activity diagrams and data flow diagrams.

3.2 Problem Analysis

3.2.1 Current System Scenario

Currently, many university students and budget-conscious individuals rely on pre-determined recipe blogs, generalized cooking videos, or manual grocery tracking. Very few digital platforms provide customized meal suggestions based on real-time pantry inventory, strict financial budgets, or localized cultural constraints.

Here are the normal limitations of current systems:

i. Manual Inventory Tracking: Users must manually type out every ingredient they possess, leading to omission, frustration, and eventual app abandonment.

ii. No Financial Context: Existing apps focus on calories but ignore the cost of ingredients, rendering them useless for users on a strict budget (e.g., under RM10/day).

iii. Cultural Blind Spots: Mainstream apps are Western-centric and lack built-in filters for Halal compliance or traditional Malaysian state cuisines.

iv. Static Algorithms: Rule-based keyword matching limits recipe discovery, leading to wasted leftover ingredients and unnecessary household food waste.

Consequently, there is a huge gap between the available technology and users' practical culinary needs.

3.2.2 Activity Diagram of Existing Conventional System

Below is a simplified Activity Diagram showing the workflow in traditional manual recipe searching and meal planning.

Figure 3.1: Activity Diagram showing traditional workflow

Figure 3.1 demonstrates the manual, traditional workflow of a user attempting to search for recipes, track their ingredients, and check their budget before cooking.

As seen above, the traditional method is linear, slow, highly susceptible to human error, and lacks the adaptability needed to reduce food waste and optimize spending simultaneously.

3.2.3 Problem Solving Approach in The Fridge Chef

The Fridge Chef system leverages a technology-first approach that solves the issues described above through several strategic components:

i. AI-Driven Personalization: Uses the Groq LLaMA-3 model for natural language synthesis, generating dynamic, culturally appropriate recipes rather than retrieving static text.

ii. CNN-Based Food Recognition: A trained Convolutional Neural Network processes user-uploaded images of their fridge/pantry to instantly identify available ingredients, removing manual input friction.

iii. OpenDOSM Integration: Calculates real-time meal costs based on the median retail prices from the Malaysia Open Data Portal to ensure strict budget compliance.

iv. Cultural Filters: Ensures all generated recipes adhere to Halal requirements and offers options for traditional Malaysian state cuisines (Masak Lemak Cili Api).

v. Waste Reduction Loop: Prioritizes recipes that utilize perishable leftovers identified by the CNN model, directly combatting domestic food waste.

3.2.4 Statistical Insights

To support the need for such a system:

i. According to global sustainability reports, household food waste accounts for a significant percentage of global greenhouse gas emissions, much of which stems from poor leftover management.

ii. In Malaysia, rising inflation has severely impacted the purchasing power of budget-conscious individuals and university students, forcing a critical need for budget-optimized nutritional planning.

3.3 Requirement Analysis

This section outlines the detailed system requirements derived from the problem and solution domain of The Fridge Chef application. It identifies the essential data, functionality, quality measures, and supporting tools that enable the system to operate efficiently, securely, and accurately.

3.3.1 Data Requirements

The data requirements involve user-input data, image processing outputs, external API fetches (OpenDOSM), and LLM-generated text (Groq API).

3.3.1.1 Types of Input and Interface

i. User registration and login credentials.

ii. Configuration settings: financial budget (e.g., RM10), caloric limits, allergies.

iii. Food image uploads for ingredient recognition (via CNN model).

iv. Selections for cultural filters (Halal, State Cuisine).

3.3.1.2 Data Stored in Database

i. User profiles and authentication data.

ii. Saved recipes and generated meal plans.

iii. Historical cost savings and caloric intake logs.

iv. CNN ingredient recognition metadata.

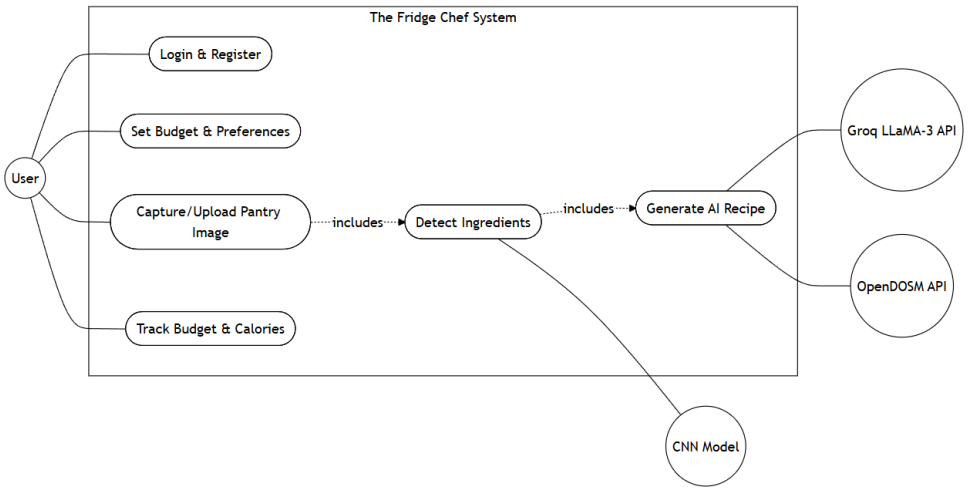


Figure 3.2: Use Case Diagram showing the User interacting with Login, Camera Upload, Recipe Generation, and Budget Tracking

Figure 3.2 shows the Use Case diagram, detailing how the end-user interacts with core system functions such as login, image uploading, and recipe generation.

3.3.1.3 Data Dictionary

Table 3.1: Data Dictionary for users Table

Field Name	Data Type	Description
id	Integer (Primary Key)	Unique auto-incrementing identifier for the user.
username	Text	Unique username for authentication.
password	Text	Cryptographically hashed user password.
phone_number	Text	User's contact number (optional).
age	Integer	User's age for BMR calculations.
gender	Text	User's gender for caloric calculations.
height	Float	User's height in cm.
weight	Float	User's weight in kg.
activity_level	Text	Physical activity multiplier string.

bmi	Float	Calculated Body Mass Index (BMI).
health_goal	Text	Defined target (e.g., Caloric Deficit, Surplus).
allergies	Text	User's dietary restrictions and allergies.
target_calories	Float	Calculated daily caloric limit.
target_protein	Float	Calculated daily protein macro target (g).
target_carbs	Float	Calculated daily carbohydrate macro target (g).
target_fat	Float	Calculated daily fat macro target (g).

Table 3.2: Data Dictionary for recipes Table

Field Name	**Data Type**	**Description**
id	Integer (Primary Key)	Unique auto-incrementing identifier for the recipe.
user_id	Integer (Foreign Key)	Associates the recipe with a specific user account.
recipe_data	Text (JSON)	Serialized JSON string containing the recipe title, cooking steps, estimated costs, and macro breakdown.
timestamp	Datetime	Exact date and time the recipe was saved by the user.

Table 3.3: Data Dictionary for shopping_list Table

Field Name	Data Type	Description
id	Integer (Primary Key)	Unique auto-incrementing identifier for the shopping item.
username	Text	Associates the item with the authenticated user.

item_name	Text	Name of the missing ingredient required for a recipe.

3.3.2 Functional Requirements

The Fridge Chef has been designed with modular, interactive functionalities that handle the flow of data from visual inputs to personalized, budget-conscious outputs.

3.3.2.1 Core Functions

1. Authentication Module: Secure login and sign-up flow for users.

2. Food Scanner Module: Allows users to upload or capture images of ingredients; the CNN model processes the image and returns a list of identified items.

3. Budget & Nutrition Engine: Fetches real-time ingredient prices from the OpenDOSM API and calculates aggregate meal costs.

4. Recipe Synthesizer: Uses the Groq LLaMA-3 API to generate detailed cooking instructions based on detected ingredients, budget, and cultural constraints.

5. Preference Flow: Collects and stores user constraints such as Halal requirements and specific Malaysian state cuisine preferences.

6. Dashboard Display: Visualizes saved recipes, budget tracking, and caloric data.

3.3.3 Non-Functional Requirements

Non-functional requirements are essential for evaluating the overall usability, reliability, and performance of the system.

Table 3.4: Non-Functional Requirements

Category	Requirement Description
Performance	The Progressive Web Application (PWA) client initialization and Document Object Model (DOM) rendering shall execute within a maximum latency threshold of 3000 milliseconds over standard 4G network conditions.
Scalability	The FastAPI backend architecture shall support asynchronous concurrency to process multiple simultaneous image inferences and LLM API requests without service degradation or bottlenecking.
Accuracy	The localized CNN ingredient classifier shall maintain a minimum validation accuracy threshold of 85%, ensuring reliable feature extraction under variable environmental lighting conditions.
Security	Cryptographic protocols (SHA-256) shall be strictly enforced for user authentication data, alongside server-side environment encapsulation (.env) for all third-party API tokens (Groq/OpenDOSM).
Usability	The graphical user interface (GUI) shall adhere to a minimalist human-computer interaction (HCI) model, requiring a maximum navigation depth of three distinct user interactions to complete the primary recipe synthesis workflow.
Availability	The system architecture shall implement robust exception handling and fallback mechanisms to ensure uninterrupted core module availability during upstream API latency or temporary outages.
Responsiveness	The total algorithmic turnaround time from user image capture to the frontend rendering of the LLM-synthesized JSON payload shall not exceed 5000 milliseconds to preserve high user retention.

3.3.4 Other Requirements

3.3.4.1 Software Requirements

i. **Python (FastAPI):** For backend application logic and model serving.

ii. **TensorFlow / Keras:** For training the custom CNN model.

iii. **Groq API (LLaMA-3):** For Natural Language Processing and recipe generation.

iv. **HTML, CSS, JavaScript (PWA):** For frontend cross-platform mobile web development.

3.3.4.2 Hardware Requirements

i. **Laptop/PC with GPU:** For training the CNN image classification model efficiently.

ii. **Smartphone with Camera:** For testing the PWA deployment and capturing real-world ingredient images.

3.3.4.3 Other Requirements

i. Internet connection (to access Groq API and OpenDOSM).

ii. Access to Kaggle or custom image datasets for training the Malaysian ingredients CNN.

iii. GitHub for version control.

3.4 Summary

This chapter has detailed the analysis phase of The Fridge Chef project, focusing on identifying the core requirements necessary for effective system

functionality and user experience. The analysis began with an overview of the current culinary challenges faced by university students and budget-conscious individuals, specifically emphasizing the limitations of conventional apps regarding budget tracking and localized cultural relevance.

The system's requirements were dissected into functional and non-functional categories, emphasizing its CNN image recognition capabilities, LLaMA-3 recipe synthesis, and OpenDOSM economic integration. Key data inputs such as financial budgets, cultural preferences, and camera uploads were identified as central to the system's logic. All software, hardware, and supporting tools were justified based on their role in delivering a responsive, intelligent, and scalable solution.

With the requirement analysis completed, the next chapter will focus on System Design, where the architectural blueprint of The Fridge Chef will be illustrated.

CHAPTER 4: DESIGN

4.1 Introduction

This chapter presents the design phase of "The Fridge Chef", an AI-Powered Culinary Assistant. It defines the transition from the analytical insights gathered in the previous chapter into a structured system blueprint. This includes both the preliminary and detailed designs of the system's architecture, interfaces, and functionalities.

The chapter elaborates on how each module, including the Convolutional Neural Network (CNN) ingredient scanner, the Groq LLaMA-3 recipe generator, the OpenDOSM financial calculator, and the user interfaces, has been designed to fulfill the functional and non-functional requirements. It also introduces key design elements such as the system architecture, navigation flow, database schema, and AI component design.High-Level Design.

4.2 High-Level Design

The structural elements and their interconnections inside the system are depicted in The Fridge Chef's high-level design. It describes the architectural structure that makes it possible for several modules to operate together, including image recognition, natural language processing, financial calculations, and user authentication.

4.2.1 System Architecture

The core functionality of The Fridge Chef revolves around the ability to recognize localized Malaysian food items from images taken by the user, and

immediately synthesize a culturally and financially appropriate recipe. This is achieved through a multi-tier architecture.

Upon launching the Progressive Web Application (PWA), the user interface communicates via asynchronous HTTP/REST requests with a Python FastAPI backend. When a user captures an image of their pantry, the image is parsed and passed to the backend where a custom-trained CNN model classifies the raw pixel data into discrete ingredient labels.

To ensure the economic viability of the meal, the backend retrieves real-time median retail prices for the identified ingredients via the Malaysia Open Data Portal (OpenDOSM) API. This localized financial data, alongside the user's predefined budget and dietary constraints (Halal), is structured into a rigid prompt and sent to the Groq API (running the LLaMA-3 model). The Large Language Model processes this contextual prompt and returns a structured recipe in JSON format, which the frontend parses and renders into an interactive recipe card.

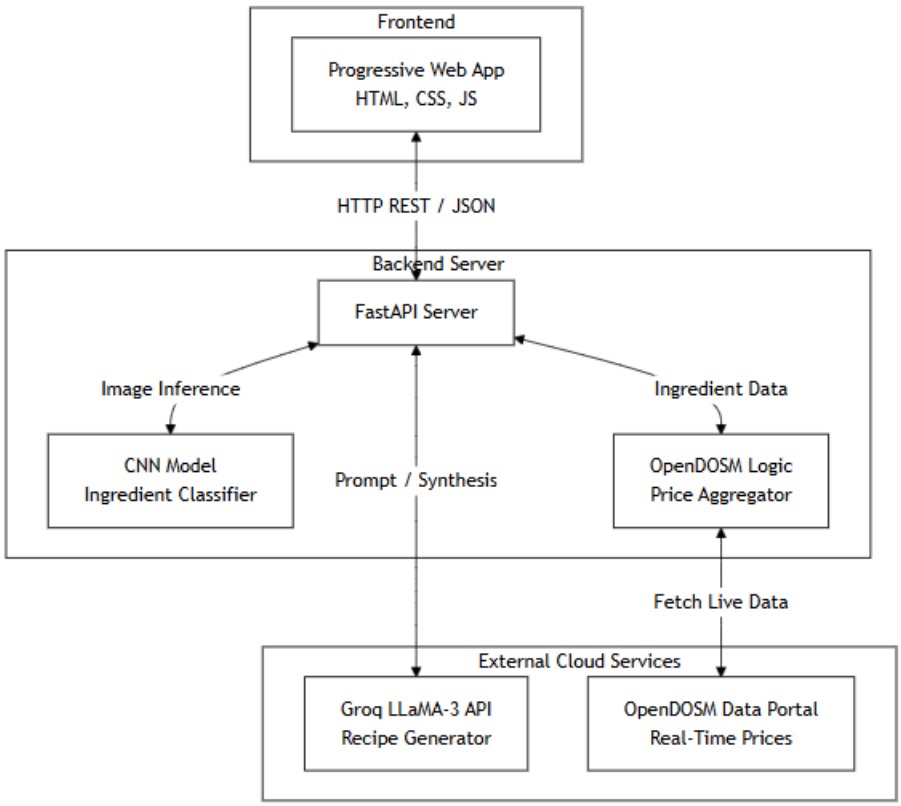


Figure 4.1 System Architecture Diagram

Figure 4.1 outlines the system architecture, showing the interaction between the Progressive Web App frontend, the FastAPI backend, and the external cloud services (Groq API and OpenDOSM).

4.2.2 User Interface Design

The user interface was developed as a Progressive Web Application (PWA) using HTML, CSS, and JavaScript. This approach ensures immediate cross-platform compatibility and allows users to "install" the app directly to their home screens without the overhead of native app store deployment.

Table 4.1: UI Components

UI Screen	**Purpose**
Login/Signup	Collects and validates user credentials via the /api/login and /api/register endpoints.
Preference Onboarding	Collects dietary restrictions (Halal), daily budget limits (RM), and caloric goals to be saved in the SQLite database.
Dashboard	Displays saved recipes, tracks current daily expenditure against the OpenDOSM limit, and provides quick-access buttons.
Ingredient Scanner	A CNN-based camera interface utilizing the device's native camera API for snapping pictures of fridge/pantry items.
Recipe Recommendation	Dynamically displays LLaMA-3 generated meal cards with exact OpenDOSM cost calculations and step-by-step cooking instructions.

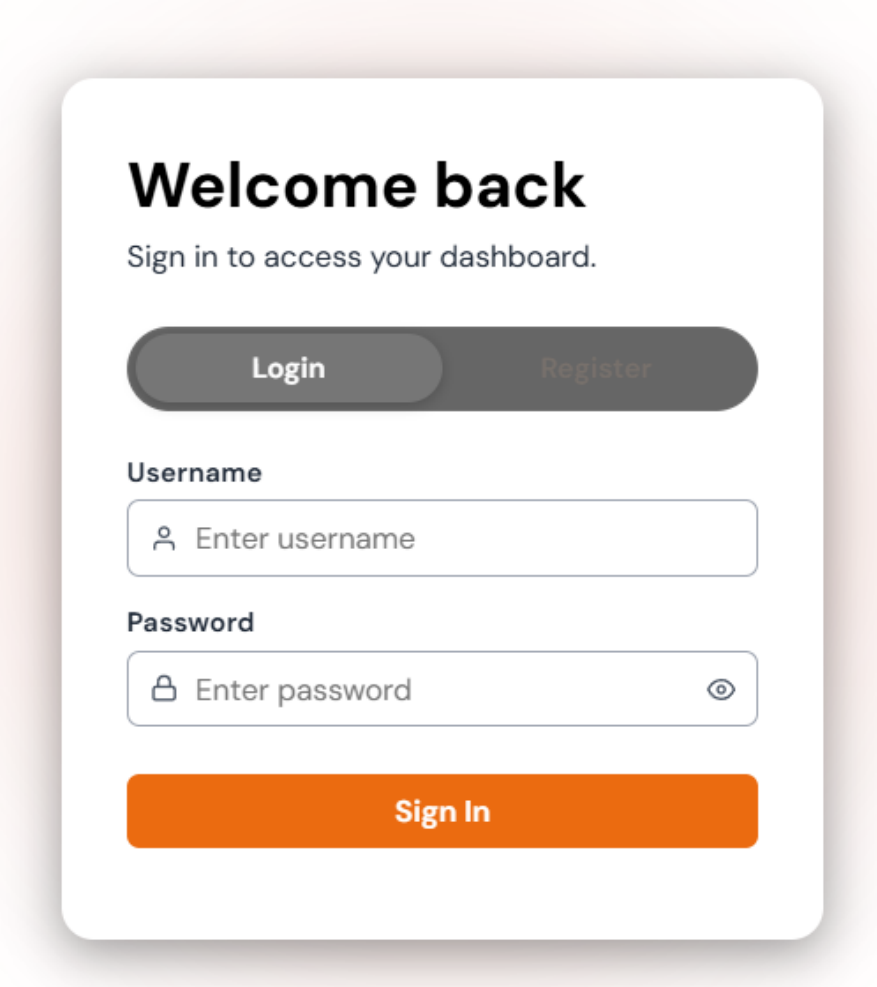


Figure 4.2: Screenshot of the Login Interface

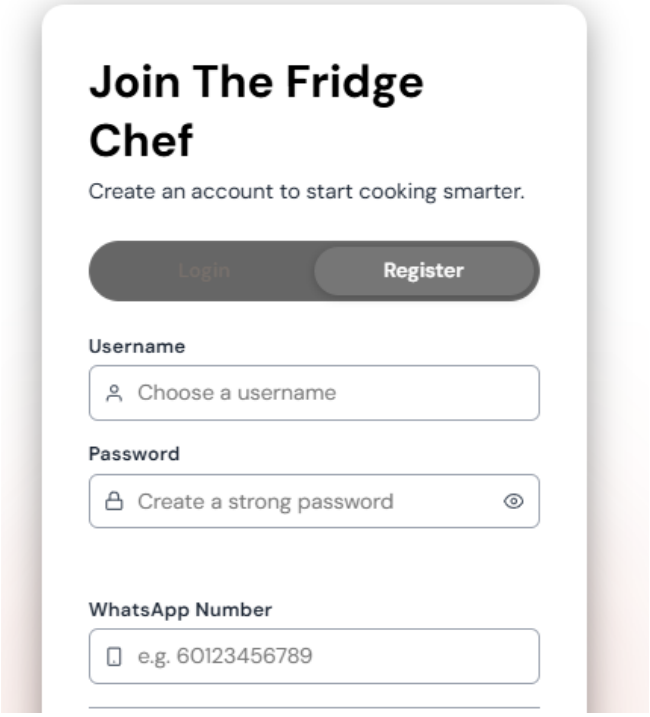


Figure 4.3: Screenshot of the Authentication Interface

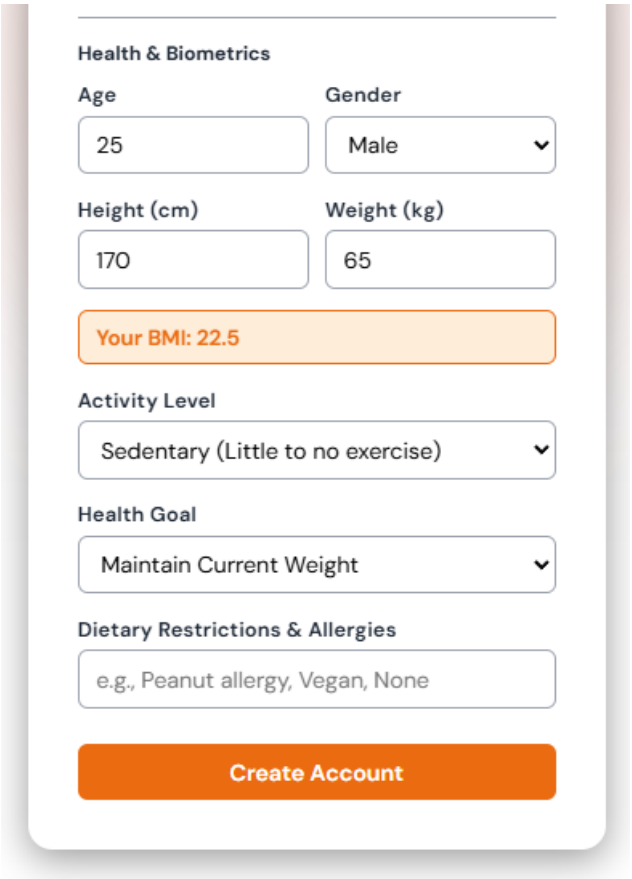


Figure 4.4: Screenshot of the Authentication Interface

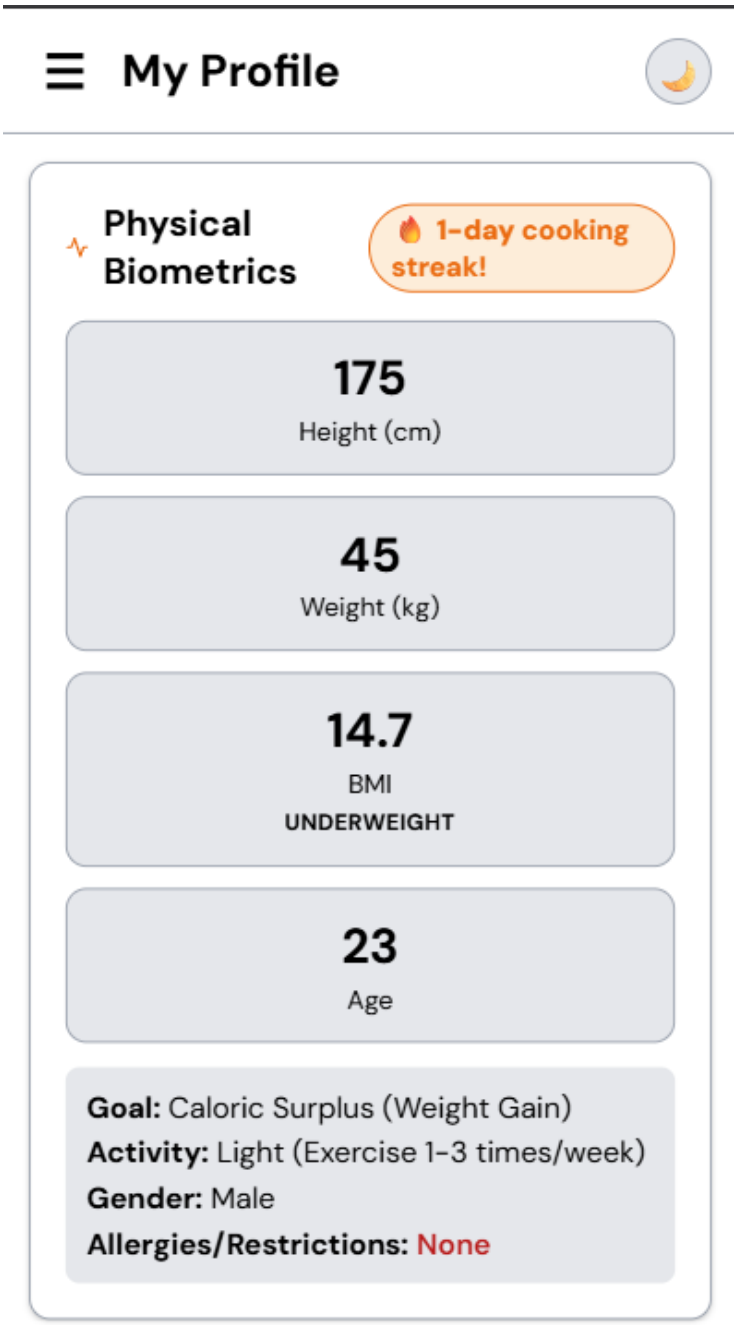


Figure 4.5: Screenshot of the User Dashboard Interface (physical Biometrics)

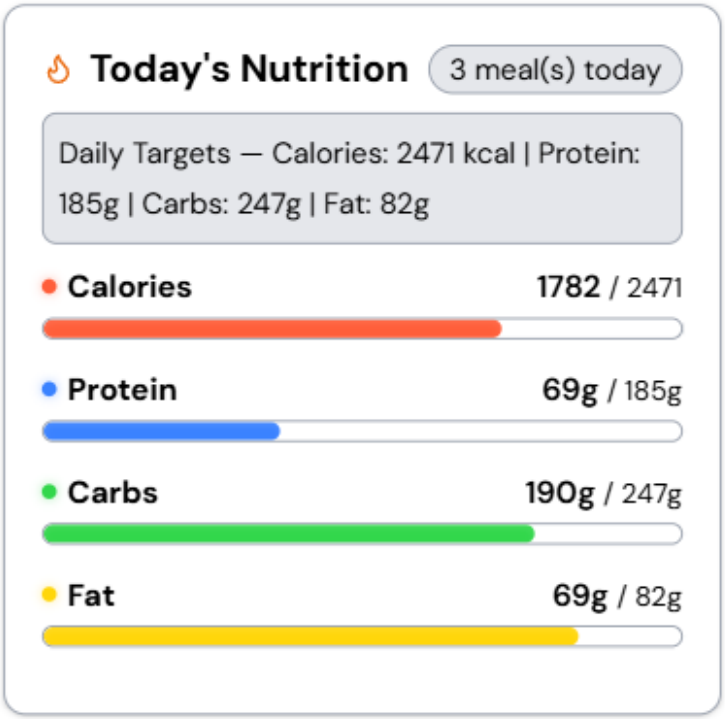


Figure 4.6: Screenshot of the User Dashboard Interface (today nutrition)

Figure 4.7: Screenshot of the User Dashboard Interface (Meal Analytics)

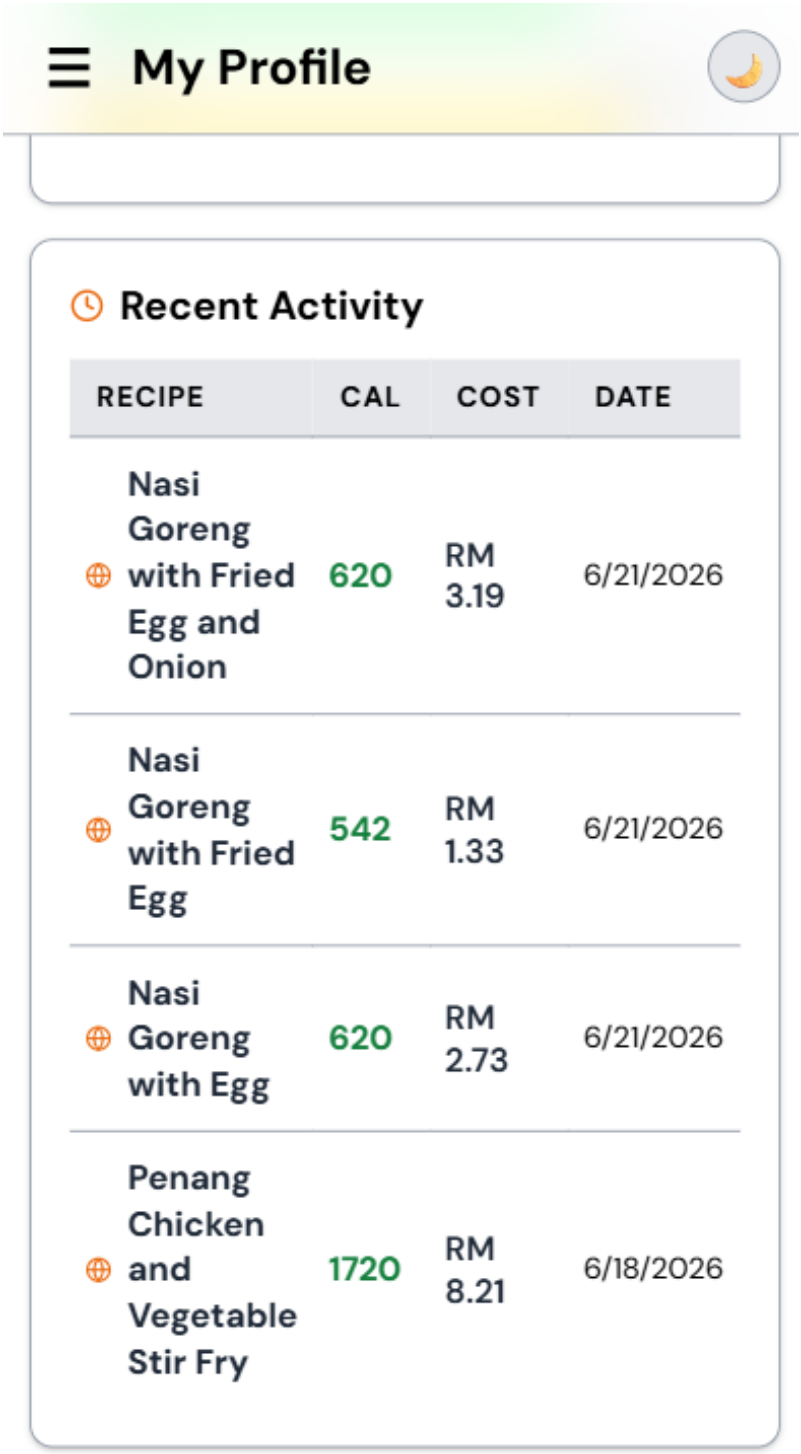


Figure 4.8: Screenshot of the User Dashboard Interface (Recent Activity)

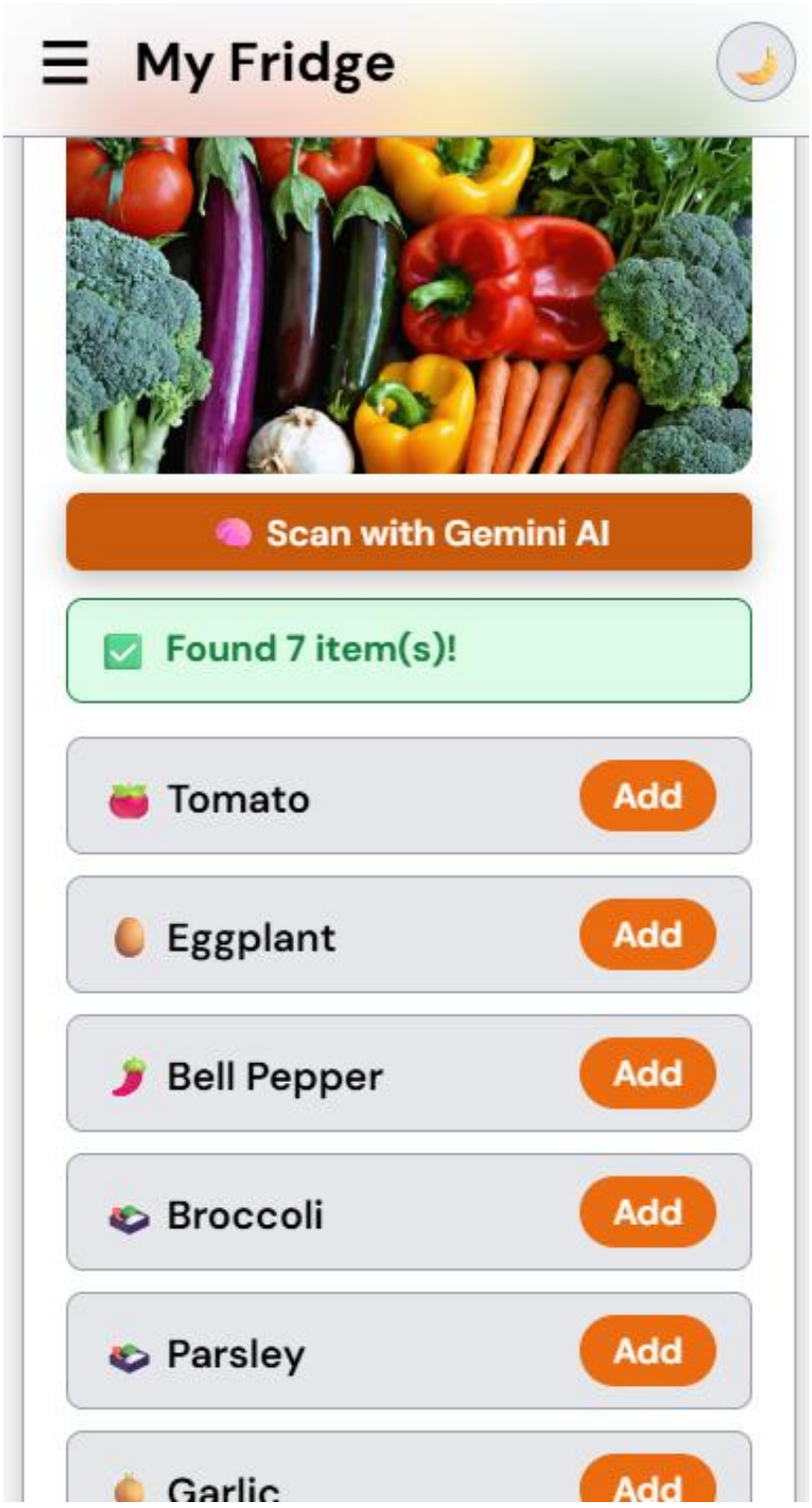


Figure 4.9: Screenshot of the CNN Ingredient Scanner Interface

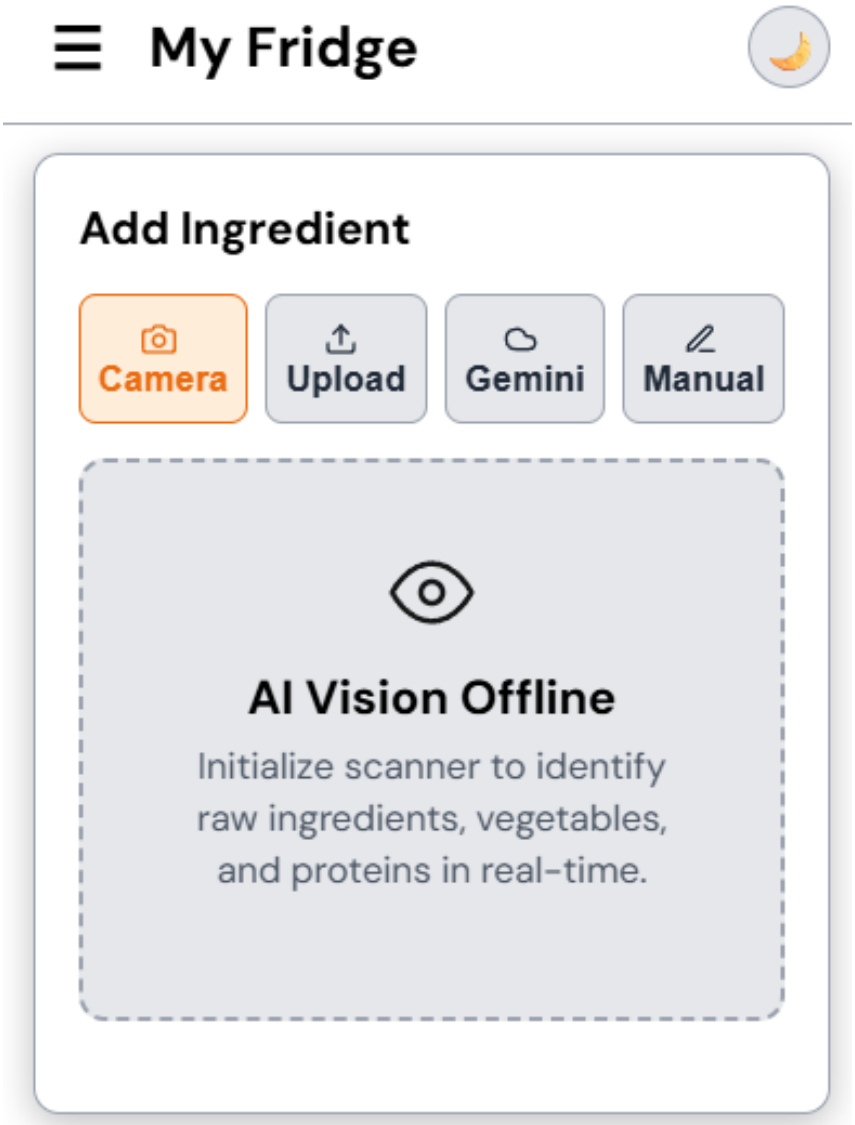


Figure 4.10: Screenshot of the CNN Ingredient Scanner Interface

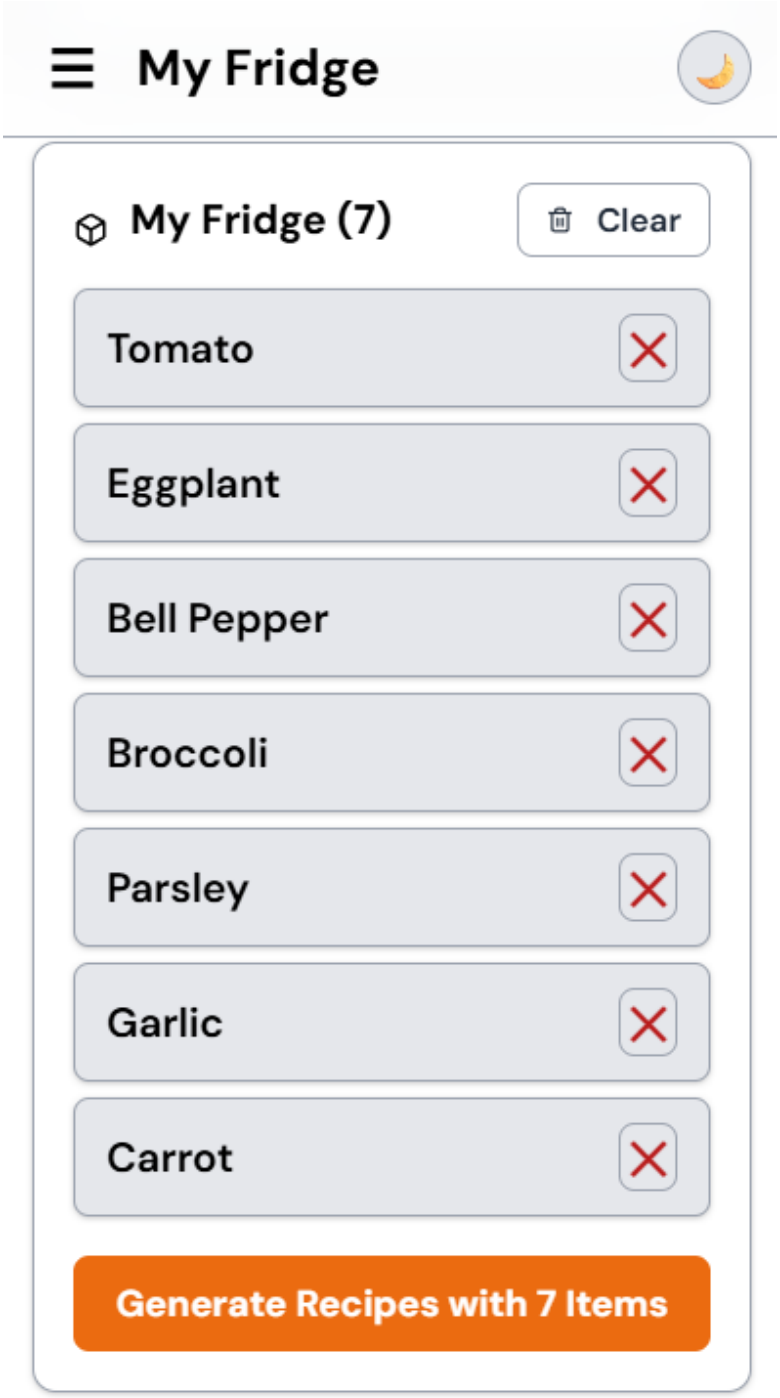


Figure 4.11: Screenshot of ingredients save in fridge

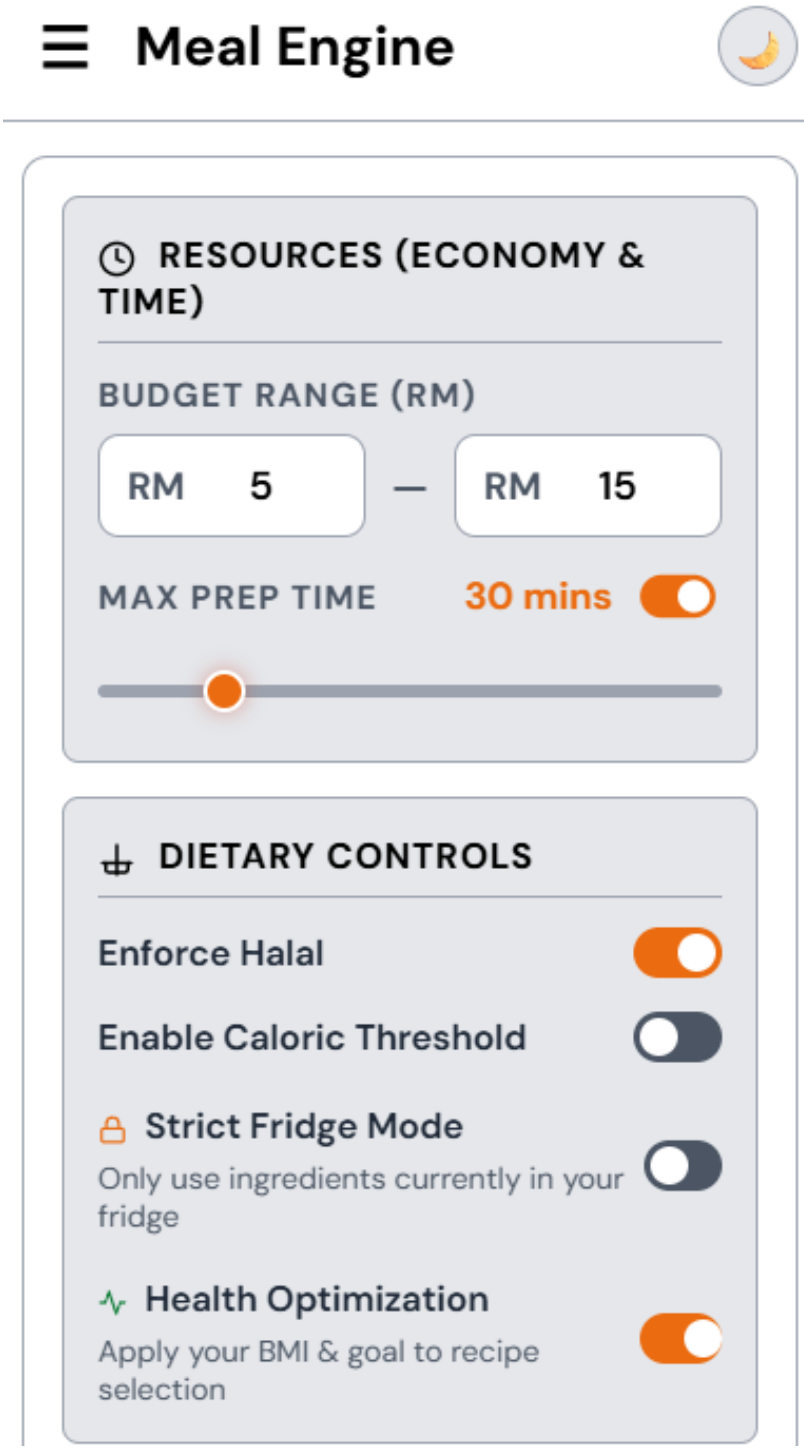


Figure 4.12: Screenshot of Meal Engine Parameter

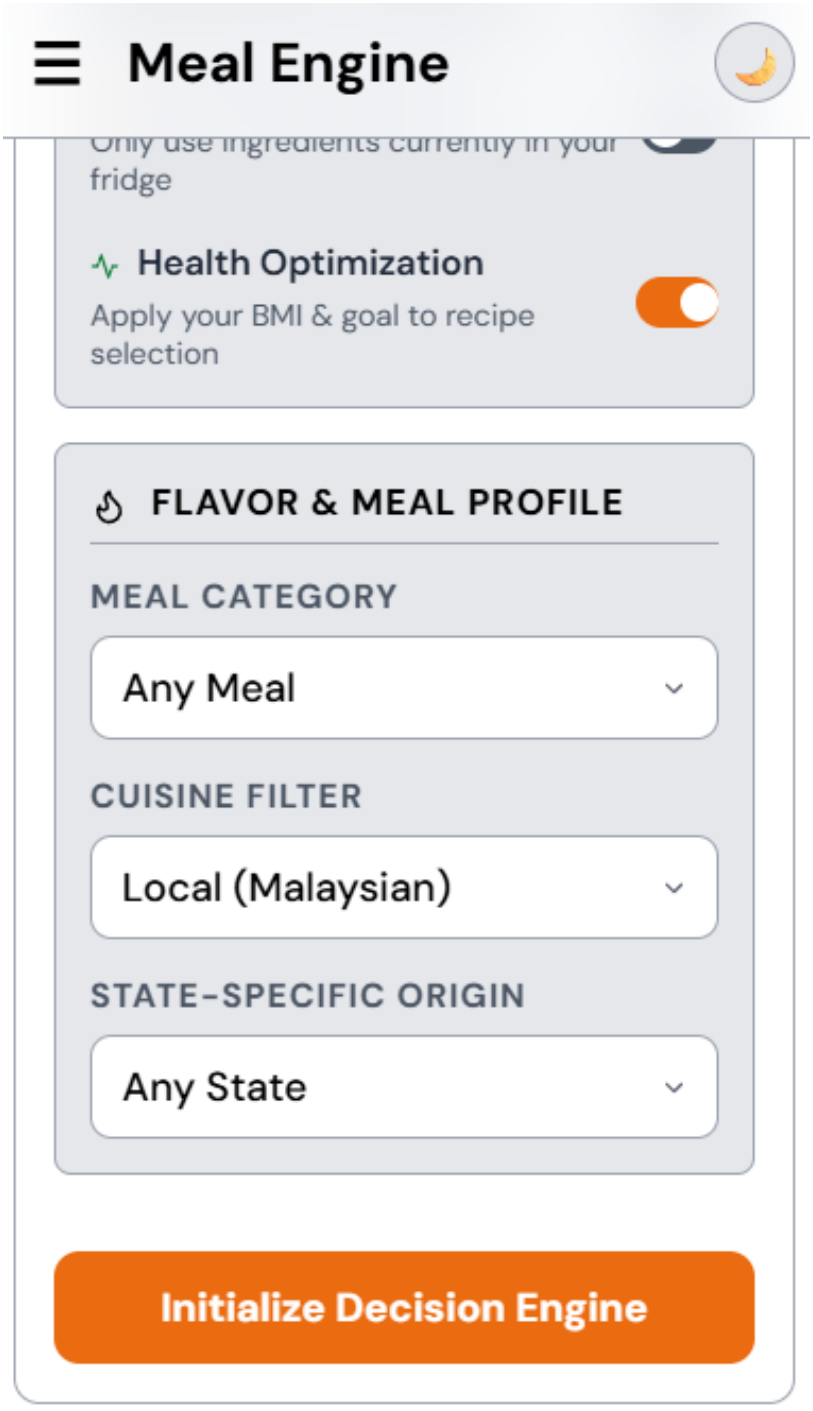


Figure 4.13: Screenshot of Meal Engine Parameter

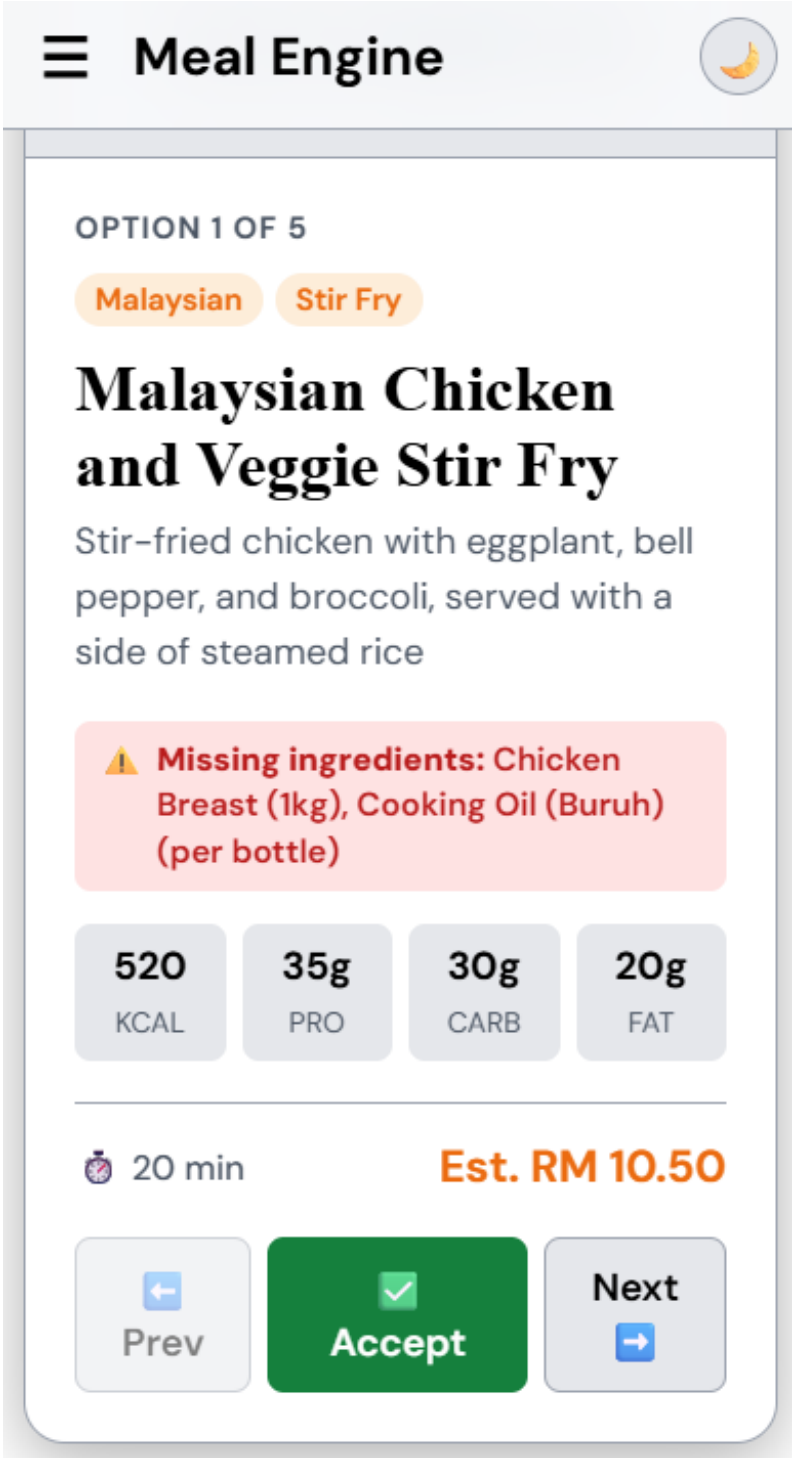


Figure 4.14: Screenshot of the Recipe Recommendation Output

4.3 Detailed Design

This section elaborates on the inner logic and structural approach of The Fridge Chef's functional components. The goal is to outline how the application works in terms of input navigation, validation, output presentation, and overall user journey.

4.3.1 Navigation Design

The Fridge Chef employs a hybrid navigation system typical of modern Single Page Applications (SPAs). The main interface consists of a persistent navigation bar that utilizes JavaScript DOM manipulation (switchTab()) to instantly swap between essential modules such as Home, Scanner, and Profile without triggering full page reloads.

4.3.1.1 Navigation Highlights

i. **Seamless Redirection & Session State:** After a successful call to /api/login, the backend returns a payload containing the user's unique ID, budget limit, and Halal constraints. This data is securely cached in the browser's localStorage. The navigation router detects this valid session and automatically redirects the user to the home dashboard, bypassing the login screen on future visits.

ii. **Linear Scanner Flow:** Capturing an image in the Scanner immediately triggers an asynchronous fetch() call to the /api/scan/cnn endpoint. The UI locks into a "Processing" state to prevent duplicate submissions, transitioning directly to the Recipe Recommendation page once the Groq API successfully returns the synthesized JSON response.

4.3.2 Input Design

User inputs in The Fridge Chef are gathered through a combination of form fields, dropdowns, and visual image uploads.

- **Text/Numeric Inputs:** Users input their daily budget (RM) and biometric data (weight, height, age) using numeric keyboards. The frontend validates these strings to prevent SQL injection or invalid character submissions before sending the payload to the FastAPI backend.

- **Visual Inputs:** The core input mechanism is the image scanner. Input validation ensures that an image file (JPEG/PNG) is captured as a multipart/form-data object and successfully uploaded via

the UploadFile class in FastAPI before the system attempts to trigger the CNN inference pipeline.

4.3.3 Output Design

The output mechanisms in The Fridge Chef are designed to give instant, personalized, and financially structured feedback based on the exact visual inputs detected. Outputs include ingredient detection labels, total meal costs, and structured cooking steps.

To prevent formatting errors, the backend forces the Groq LLaMA-3 model to return outputs in a strict JSON object format. The frontend parses this JSON payload to display a stylized recipe card containing:

- **Recipe Title:** A dynamically generated, culturally relevant dish name.
- **Total Estimated Cost (RM):** The exact cost of the meal, deterministically aggregated by the OpenDOSM pipeline (bypassing the LLM to prevent arithmetic hallucination).
- **Macronutrient Breakdown:** Estimated calories, proteins, and fats.
- **Step-by-step Instructions:** Clear, bulleted cooking protocols.

4.4 Database Design

The Fridge Chef data is persistently stored and managed using a lightweight relational database (SQLite) connected via the FastAPI backend. To ensure the integrity and confidentiality of user data, the application implements strict cryptographic security measures. User passwords are **never** stored as plain text. Instead, the system utilizes the **SHA-256 cryptographic hashing algorithm** (via Python's hashlib library). Upon registration, the user's password is irreversibly converted into a 64-character hexadecimal signature before being committed to the SQLite database. During authentication, the system hashes the inputted password and compares it against the stored signature, ensuring complete data privacy and rendering the database immune to plain-text credential leaks.

Table 4.2: Database Entities and Their Purposes

Entity/Table	Fields	Purpose
Users	id, username, password, phone_number, age, gender, height, weight, activity_level, bmi, health_goal, allergies	Stores user biometric profiles, login credentials, and health goals.
History	log_id, user_id, recipe_name, calories, protein, carbs, fat, cost_rm, timestamp	Logs daily meals, macro intake, and exact RM costs for budget tracking.
SavedRecipes	id, username, recipe_name, ingredients, instructions	Stores detailed, AI-generated recipes that the user wants to keep for future cooking.
ShoppingList	id, username, recipe_name, item_name	Tracks individual missing ingredients the user needs to purchase from the store.

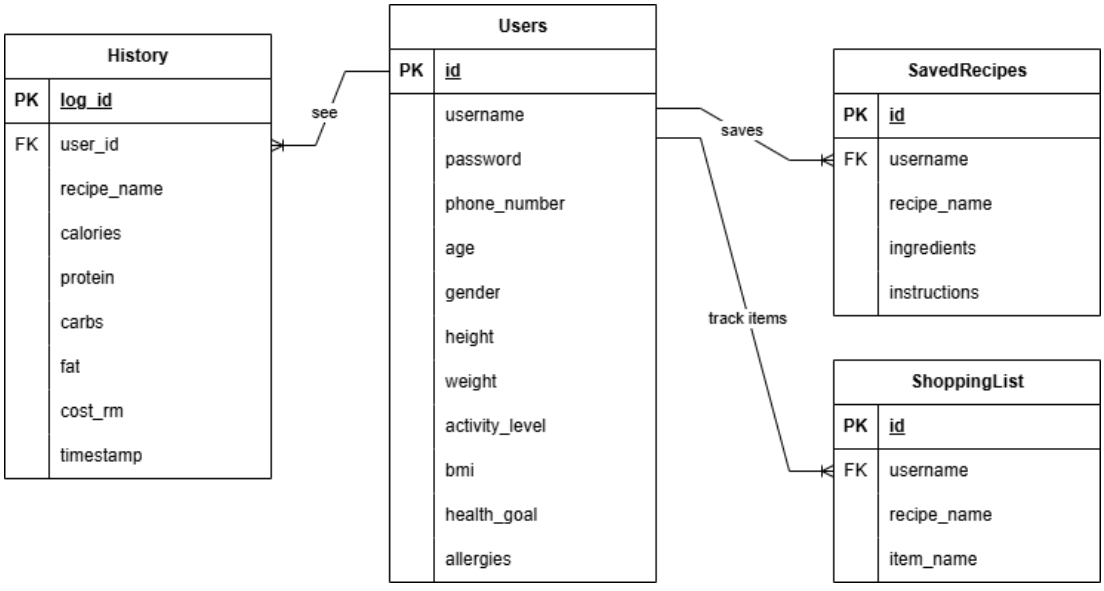


Figure 4.15: Entity-Relationship Diagram (ERD)

This ERD visually shows how your three core tables are linked using Primary Keys (PK) and Foreign Keys (FK). Evaluators look for this specifically to confirm your database is relational.

4.5 AI Component Design

The AI components in The Fridge Chef were carefully architected to deliver personalized dietary guidance using two primary technologies: a custom-trained Convolutional Neural Network (CNN) for visual ingredient recognition, and a cloud-based Generative Pre-trained Transformer (LLaMA-3) for recipe synthesis.

4.5.1 Dataset Design and Characteristic

The core training dataset for the image classification model was built to specifically recognize 22 common localized Malaysian food ingredients (e.g., eggs, red onions, tomatoes, poultry, local leafy greens).

All images were resized to 224x224 pixels, standardized into RGB format, and augmented using horizontal flipping, random rotation, and zooming during preprocessing. This augmentation was essential to improve the model's ability to generalize and reduce overfitting, given the real-world variability in lighting and camera angles when users take photos inside their refrigerators.

4.5.2 AI Techniques Employed

4.5.2.1 Convolutional Neural Network (CNN) For Ingredient Classification

CNNs are designed to process grid-like data such as images. The system's CNN architecture comprises convolutional layers to extract hierarchical features, pooling layers to downsample, dropout layers for regularization, and fully connected layers leading to a 22-class softmax output layer.

In production, this is deployed via the /api/scan/cnn endpoint. The Python preprocess_image() function standardizes the uploaded image into a NumPy array and feeds it into the optimized TensorFlow Lite (.tflite) interpreter. The system extracts the probabilities from the Softmax output layer, using argmax to determine the highest confidence index, which is deterministically mapped to the predefined CLASS_NAMES array ("Tomato" or "Chicken").

4.5.2.2 LLaMA-3 (Groq API) For Recipe Generation

The Groq API provides ultra-low latency access to the LLaMA-3 model, which powers the Natural Language Processing module. Using highly structured, programmatic prompts, it acts as a culinary engine. LLaMA-3 excels at following complex system prompts: it takes the exact array of CNN-detected ingredients, verifies them against the user's saved Halal and allergy requirements, cross-references the deterministic OpenDOSM financial limits, and synthesizes a culturally appropriate recipe without exceeding the budget.

4.5.3 System Design Logic and AI Flow

The AI logic was developed in specific layers, ensuring absolute modularity and interaction between the image model, the deterministic financial API, and the LLM.

Table 4.3: System Design Logic

Component	Trigger	Internal Processing	Output
CNN Model	User uploads pantry image	Image is standardized and passed through convolutional layers via TFLite.	Top predicted ingredient label and confidence score.
OpenDOSM Logic	Ingredients identified	Exact median retail prices are fetched and cross-referenced with item categories.	Array of exact ingredient costs (RM).
Groq LLaMA-3	Prices & Ingredients compiled	Structured prompt is engineered to enforce JSON output and strict budget constraints.	JSON-formatted cooking instructions and macronutrients.

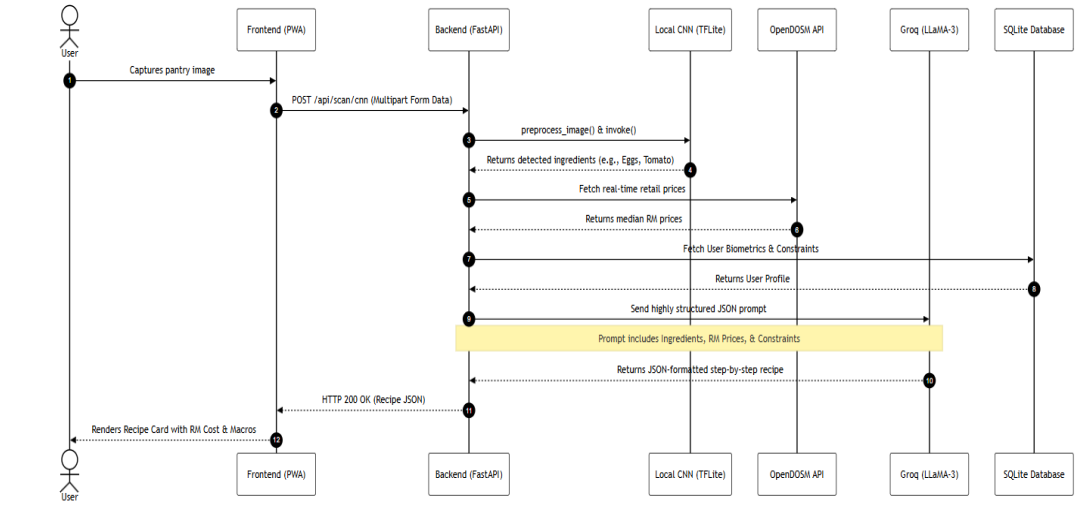


Figure 4.16: System Sequence Diagram

Figure 4.12 maps the chronological data flow and system interactions required to generate a customized meal plan. The sequence initiates when the user captures a pantry image via the Progressive Web Application (PWA) frontend. The image payload is transmitted to the FastAPI backend, which invokes the local Convolutional Neural Network (TFLite) to extract and classify the ingredients. The backend then deterministically fetches real-time median retail prices for these ingredients via the OpenDOSM API, and retrieves the user's specific biometric constraints (Halal requirements, caloric limits) from the SQLite database. Finally, this aggregated context is structured into a rigid prompt and sent to the Groq LLaMA-3 module, which synthesizes a mathematically accurate, JSON-formatted recipe that is returned and rendered on the user's dashboard.

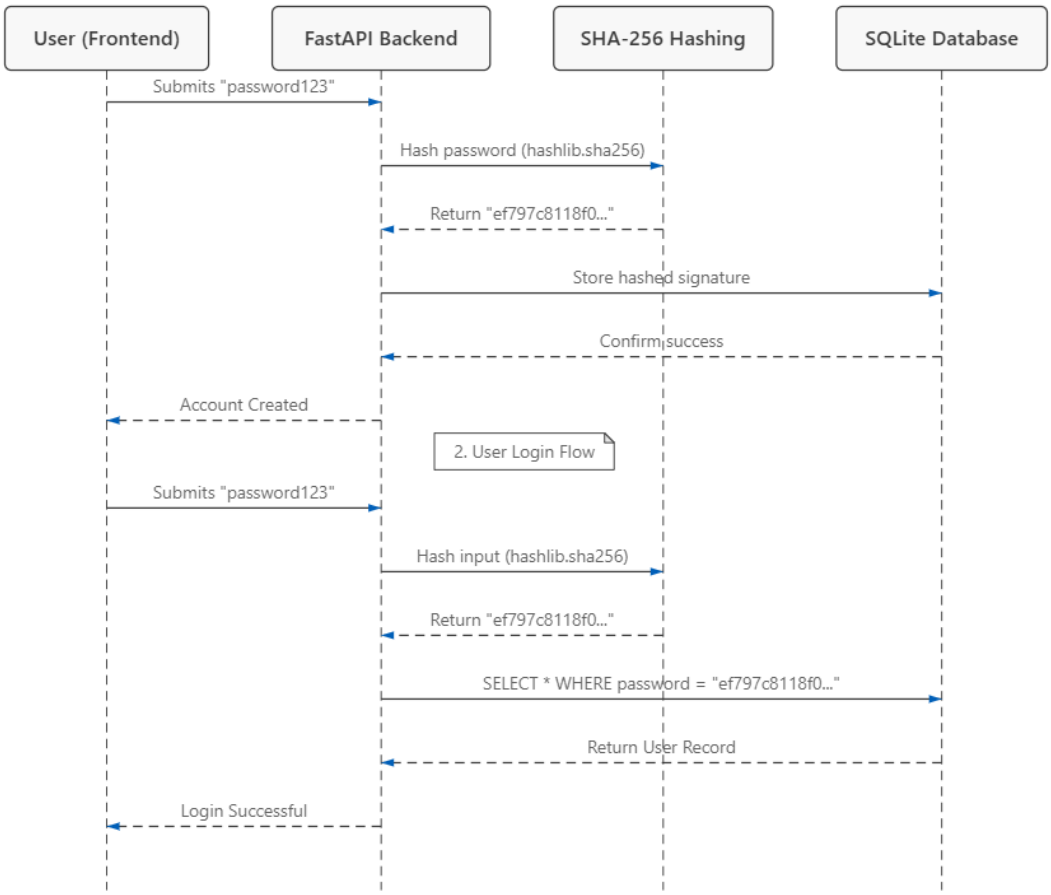


Figure 4.17: Secure Authentication Flow (SHA-256)

Figure 4.13 illustrates the chronological sequence of data exchanges during the user registration and login processes. When a user submits their credentials via the frontend interface, the plaintext password is intercepted by the FastAPI Backend and immediately routed through the SHA-256 cryptographic hashing function. This ensures that only the resulting 64-character hexadecimal signature is transmitted to and stored within the SQLite Database. During subsequent login attempts, the system hashes the provided input and queries the database to match the signature. This architectural flow guarantees that sensitive user credentials are mathematically obfuscated at all times, preventing unauthorized data exposure.

4.5.4 System Functionality Summary

i. **Visual Component:** The CNN classifier processes unstructured visual food images, translating them into structured text arrays.

ii. **Economic Component:** Deterministic backend logic maps those recognized ingredients to real-world financial values, preventing algorithmic hallucination.

iii. **Generative Component:** The LLaMA-3 model synthesizes the final output, ensuring high personalization, strict budget compliance, and cultural relevance.

4.6 Summary

This chapter outlined the comprehensive technical design of The Fridge Chef system, detailing both high-level architecture and code-level component interactions. The application was strictly designed to integrate multiple intelligent modules the CNN image scanner, OpenDOSM financial calculator, and LLaMA-3 recipe generator into a seamless Progressive Web Application. Each component's logic was mapped through user interface layouts, navigation routers, and the relational database schema.

By thoroughly dissecting the AI design logic, particularly the preprocess_image inference pipeline and the LLM prompt engineering constraints, the operational mechanics of the system have been clearly established. The next phase of the project will focus on the deployment and testing of these integrated components.

REFERENCES

Buchwalow, I. B., and Böcker, W. (2010). Immunohistochemistry: basics and methods. Berlin: Springer Verlag.

Nor Azlan Shah, M. I., Mohd Sabri, N., Tan, G. J., & Zhang, Z. (2025). CNN Integrated Mobile Application: Food Image Recognition for Recipe Generation. Journal of Computing Research and Innovation, 10(2), 198-215.

Salim, N. O. M., Zeebaree, S. R. M., Sadeeq, M. A. M., Radie, A. H., Shukur, H. M., & Rashid, Z. N. (2021). Study for Food Recognition System Using Deep Learning. Journal of Physics: Conference Series, 1963, 012014.

Chen, Y. C., & Chiang, H. C. (2025). Deep learning-based automatic food identification with numeric label. Multimedia Tools and Applications.

Anand, L., Tyagi, R., & Mehta, V. (2024). Food recognition using deep learning for recipe and restaurant recommendation. In Cyber Security and Intelligent Systems (pp. 1056). Springer.

Boyd, L., Nnamoko, N., & Lopes, R. (2024). Fine-grained food image recognition: A study on optimising convolutional neural networks for improved performance. Journal of Imaging, 10(6), 126.

Anitha, E., Nandhini, S., Palani, H. K., & Marshal, M. C. (2023). Recipe recommendation using image classification. In Proceedings of the 2023 5th International Conference on Smart City Applications (pp. 1023-1033).

Swain, M., Manyatha, A. R., Dinesh, A. S., Sampatrao, G. S., & Soni, M. (2023). Ingredients to recipe: A YOLO-based object detector and recommendation system via clustering approach. In Proceedings of the 3rd International Conference on Artificial Intelligence and Smart Energy (pp. 1397-1404).

Ciocca, G., Micali, G., & Napoletano, P. (2020). State recognition of food images using deep features. IEEE Access, 8, 32003-32017.

Pan, L., Pouyanfar, S., Chen, H., Qin, J., & Chen, S.-C. (2017). Deepfood: Automatic multi-class classification of food ingredients using deep learning. In 2017 IEEE 3rd international conference on collaboration and internet computing (CIC) (pp. 181-189).

Aguilar, E., Bolaños, M., & Radeva, P. (2017). Food recognition using fusion of classifiers based on CNNs. In International Conference on Image Analysis and Processing (pp. 213-224).

Abadi, M., Barham, P., Chen, J., Chen, Z., Davis, A., Dean, J., ... & Zheng, X. (2016). TensorFlow: A system for large-scale machine learning. *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*, 265-283.

Biørn-Hansen, A., Majchrzak, T. A., & Grønli, T. M. (2017). Progressive web apps: The possible web-native unifier for mobile development. *Proceedings of the 13th International Conference on Web Information Systems and Technologies (WEBIST)*, 344-351.

Department of Statistics Malaysia (DOSM). (2024). *Malaysia Open Data Portal - Consumer Price Index and Retail Prices*. Retrieved from https://data.gov.my/

Food and Agriculture Organization of the United Nations (FAO). (2022). *The State of Food Security and Nutrition in the World: Repurposing food and agricultural policies to make healthy diets more affordable*. Rome: FAO.

Google. (2024). *TensorFlow Lite for Mobile and Edge Devices*. Retrieved from https://www.tensorflow.org/lite

Groq. (2024). *Groq LPU Inference Engine API Documentation*. Retrieved from https://groq.com/docs

Kotu, V., & Deshpande, B. (2019). *Data Science: Concepts and Practice* (2nd ed.). Morgan Kaufmann.

LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *Nature*, 521(7553), 436-444.

Martin, R. C. (2009). *Agile Software Development, Principles, Patterns, and Practices*. Prentice Hall.

MDN Web Docs. (2024). *Progressive Web Apps (PWAs) - Core Concepts and Architecture*. Mozilla. Retrieved from https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps

Min, W., Jiang, S., Liu, L., Rui, Y., & Jain, R. (2019). A survey on food computing. *ACM Computing Surveys (CSUR)*, 52(5), 1-36.

Ramírez, S. (2024). *FastAPI Documentation: High-performance REST APIs with Python*. Retrieved from https://fastapi.tiangolo.com/

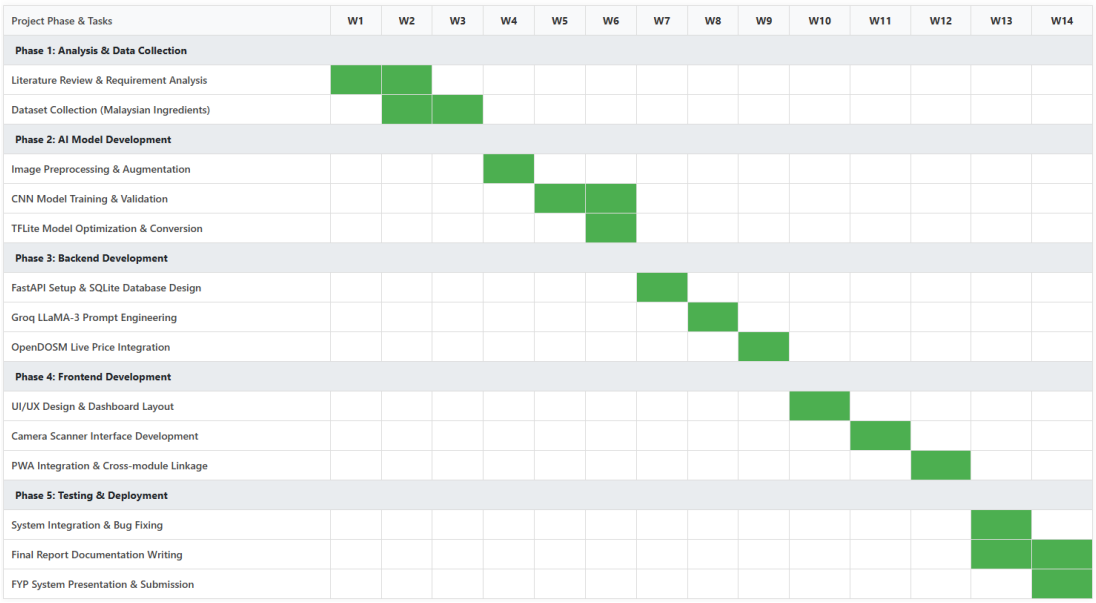
Sommerville, I. (2015). *Software Engineering* (10th ed.). Pearson.

Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M. A., Lacroix, T., ... & Lample, G. (2023). LLaMA: Open and Efficient Foundation Language Models. *arXiv preprint arXiv:2302.13971*.

Van Rossum, G., & Drake, F. L. (2009). *Python 3 Reference Manual*. CreateSpace.

Wirth, R., & Hipp, J. (2000). CRISP-DM: Towards a standard process model for data mining. *Proceedings of the 4th International Conference on the Practical Applications of Knowledge Discovery and Data Mining*, 29-39.

APPENDICES



Appendix A: Gantt chart for PSM1 milestones timeline